KartNova Analytics

The Complete Beginner's Guide

From **zero knowledge** to explaining an end-to-end Python · SQL · Power BI/DAX analytics project in an interview, and handling scenarios you have never seen.

What you will be able to do after reading
1. Explain every number, chart, query and DAX measure in the project in plain language
2. Re-derive the key ideas (RFM, cohorts, churn model, p-values, LTV:CAC, window functions, filter context) on a whiteboard
3. Answer 'what would you do if...' questions on new datasets using a repeatable problem-solving framework
4. Defend the project honestly: what it proves, what it assumes, what you would change with real data

Companion to the project folder `KartNova_Analytics/` *(data · Python · SQL · Power BI). All numbers in this guide are taken from the project's own outputs.*

Contents

How to use this guide 6

Suggested reading path . . . 6

The mindset interviewers are testing . . . 6

1. Data analytics from zero 7

1.1 What is data analytics? . . . 7

1.2 The four levels of analytics . . . 7

1.3 Who is who in a data team . . . 8

1.4 The life-cycle of an analytics project . . . 8

2. Business metrics you must know 9

2.1 Money metrics . . . 9

2.2 Order and customer metrics . . . 9

2.3 Marketing metrics . . . 10

2.4 Retail concepts . . . 10

3. Data basics: tables, keys, types and the statistics you need 11

3.1 What is a table? . . . 11

3.2 Keys and relationships . . . 11

3.3 Fact tables vs dimension tables (the star schema) . . . 11

3.4 Data types . . . 12

3.5 Statistics every analyst must know . . . 12

4. The KartNova project: the big picture 14

4.1 The story . . . 14

4.2 The data: 7 tables . . . 14

4.3 The pipeline . . . 14

4.4 Business rules (the single source of truth) . . . 15

4.5 Project folder map . . . 15

4.6 Headline results (memorise these) . . . 16

5. Python for analysts: pandas in one chapter 17

5.1 The ten operations . . . 17

5.2 The project's core helper: `build_fact()` . . . 19

6. Data cleaning: trusting your data 20

6.1 What we found and what we did . . . 20

6.2 Why each fix was chosen . . . 20

6.3 Code pattern: detect → count → fix → log . . . 21

6.4 Validation checks after cleaning . . . 21

7. Executive KPIs and exploratory analysis (EDA) 22

7.1 The KPI scorecard . . . 22
7.2 Trend and seasonality . . . 22
7.3 Category economics: revenue ≠ profit . . . 23
7.4 Pareto (80/20) analysis . . . 23
7.5 Operations: regions and payment . . . 24
8. Customer segmentation: RFM and K-Means 25
8.1 Why segment customers? . . . 25
8.2 RFM defined . . . 25
8.3 Scoring with quintiles . . . 25
8.4 From scores to named segments . . . 25
8.5 What we found . . . 26
8.6 K-Means clustering: letting the data find groups . . . 26
9. Cohort retention analysis 28
9.1 How to read a cohort table . . . 28
9.2 How the code builds it . . . 28
9.3 What we found . . . 29
9.4 Retention by acquisition channel . . . 29
10. Churn prediction: machine learning from zero 31
10.1 The business problem . . . 31
10.2 Machine learning in plain English . . . 31
10.3 The most important rule: avoid data leakage . . . 31
10.4 The features we built . . . 32
10.5 The two models . . . 32
10.6 How to judge a classifier . . . 33
10.7 Results . . . 33
10.8 From probability to money: revenue at risk . . . 34
11. Marketing efficiency: CAC, LTV and payback 36
11.1 The three numbers . . . 36
11.2 Why only 'mature' customers for LTV . . . 36
11.3 Results . . . 36
12. Market basket analysis 38
12.1 Three metrics . . . 38
12.2 Results (sub-category level) . . . 38
13. Statistics and hypothesis testing 40
13.1 The logic, step by step . . . 40
13.2 Errors . . . 40
13.3 Choosing the test . . . 41

13.4 A hand-calculable chi-square example . . . 41
13.5 Confidence intervals and the bootstrap . . . 41
13.6 Our six tests . . . 41
13.7 What the results mean for the business . . . 42
13.8 The discount-cap what-if . . . 43
14. Revenue forecasting 44
14.1 Anatomy of a time series . . . 44
14.2 Our three candidate models . . . 44
14.3 Choosing by backtest, not by fit . . . 44
14.4 The forecast . . . 45
15. SQL from zero, and every query in the project 46
15.1 The anatomy of a query . . . 46
15.2 Essential building blocks . . . 46
15.3 JOINs: combining tables . . . 47
15.4 CTEs: readable multi-step logic . . . 47
15.5 Window functions: the senior-analyst skill . . . 47
15.6 The project's database design . . . 48
15.7 Dialect differences (SQLite vs the others) . . . 48
15.8 The project queries, one by one . . . 49
15.9 Classic SQL interview patterns (with the answer idea) . . . 52
15.10 Performance basics . . . 52
16. Power BI and DAX from zero 54
16.1 What is Power BI? . . . 54
16.2 The Power BI workflow . . . 54
16.3 Power Query (M): prepare the data . . . 54
16.4 The data model . . . 55
16.5 DAX fundamentals (read this twice) . . . 56
16.6 The project's measure library, by family . . . 58
16.7 DAX queries (DAX Query View) . . . 59
16.8 Designing the dashboard . . . 60
16.9 Governance and performance . . . 61
16.10 Common DAX errors and fixes . . . 61
17. Insights and recommendations 62
17.1 Caveats to volunteer . . . 62
18. The scenario playbook: answering questions you have never seen 64
18.1 The universal method (use it out loud) . . . 64
18.2 Decomposition trees: break any metric into drivers . . . 64

18.3 Which technique for which problem? . . . 65
18.4 The same toolkit in other industries . . . 65
18.5 A/B testing in five minutes . . . 66
18.6 Fourteen worked scenarios . . . 66

19. Interview question bank with model answers 70

19.1 About the project . . . 70
19.2 Python and pandas . . . 70
19.3 SQL . . . 71
19.4 Power BI and DAX . . . 72
19.5 Statistics . . . 73
19.6 Machine learning . . . 73
19.7 Business and behavioural . . . 74

20. Cheat sheets 75

20.1 SQL . . . 75
20.2 pandas . . . 75
20.3 DAX . . . 75
20.4 Statistics and metrics formulas . . . 76
20.5 Which chart? . . . 76

21. Study plan, project pitches and glossary 77

21.1 A 14-day study plan . . . 77
21.2 The 60-second pitch . . . 77
21.3 The 3-minute structure . . . 77
21.4 STAR template for behavioural questions . . . 77
21.5 Things never to say . . . 78
21.6 Glossary (A-Z) . . . 78

How to use this guide

This guide assumes **you know nothing**. No statistics, no coding, no business vocabulary. Every chapter follows the same pattern so you always know where you are:

Box you will see	What it means
ANALOGY	A real-life comparison so the idea feels obvious before the jargon arrives
★ KEY IDEA	The one sentence to remember. If you remember nothing else, remember these
◆ IN THIS PROJECT	How the idea shows up in KartNova, with the real numbers we found
▶ WORKED EXAMPLE	A tiny hand-calculable example so you can verify the logic yourself
⚠ COMMON MISTAKE	Errors beginners (and some professionals) make; interviewers love to probe these
INTERVIEW ANGLE	How the topic is asked and how to answer

Suggested reading path

If you have...	Do this
2 weeks	Read Chapters 1-4 slowly, then 5-14 once. Do the 'Try it yourself' items. Read Chapters 18-19 the last two days.
1 week	Chapters 1-4 (fast), then pick: 7, 8, 10, 11, 13, 15, 16, then 18-19.
2 days (emergency)	Chapter 4 (project story), Chapter 17 (insights), Chapter 18 (scenario playbook), Chapter 19 (Q&A), Chapter 21 (60-second pitch).

The mindset interviewers are testing

A senior analyst is not someone who knows more functions. They are someone who (1) asks the right business question, (2) is paranoid about data quality, (3) chooses the simplest method that answers the question, (4) can prove their numbers are right, and (5) ends with a decision and a rupee impact. This guide trains exactly those five habits. Watch for them in every chapter.

> **KEY IDEA**
>
> Tools change every few years. The **thinking pattern** (question → metric → data → method → validate → recommend) never changes. Master the pattern and any new tool or scenario becomes easy.

1. Data analytics from zero

1.1 What is data analytics?

Data analytics is the practice of using data to answer questions and support decisions. A shop owner who looks at last month's sales to decide how much stock to order is already doing analytics, just without a computer. A data analyst does the same thing with millions of rows, repeatable code and statistical care.

> **ANALOGY**
>
> Think of a doctor. The patient (the business) says 'I feel unwell' (revenue is flat). The doctor orders tests (collects data), reads the results (analysis), diagnoses the cause (insight), and prescribes treatment (recommendation). Analytics is the doctor's job, applied to a business.

1.2 The four levels of analytics



Figure 1.1 - Analytics ladder: each level answers a harder question and is worth more to a business.

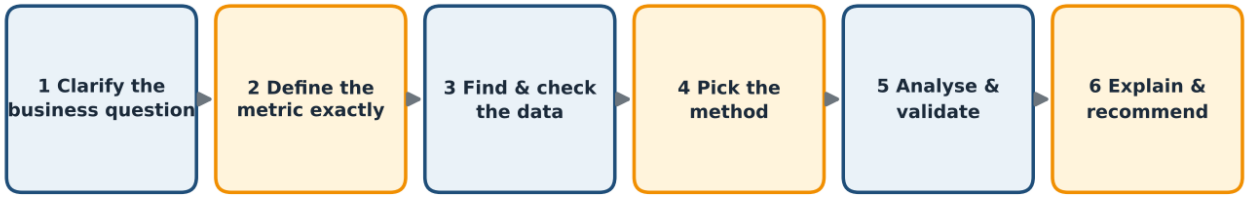
Level	Question	Typical tools	Example in this project
Descriptive	What happened?	Totals, averages, trend charts, dashboards	Net revenue was ₹13.4 Cr in 2025, up 50%
Diagnostic	Why did it happen?	Drill-downs, comparisons, hypothesis tests, cohorts	COD orders cancel 8.9% vs 3.3% for prepaid orders
Predictive	What will happen?	Machine learning, forecasting	A churn model ranks customers by risk; a 6-month revenue forecast
Prescriptive	What should we do?	Scenario simulation, optimisation, A/B tests	Capping discounts at 10% would add ≈ ₹83.9 lakh profit (if volume holds)

1.3 Who is who in a data team

Role	Main job	Main tools
Data Analyst (you)	Turn data into insights and decisions; dashboards; ad-hoc analysis	SQL, Excel, Power BI/Tableau, Python
Data Engineer	Builds the pipelines that move and store data reliably	SQL, Python/Spark, cloud, Airflow
Data Scientist	Builds predictive models and runs experiments	Python/R, statistics, ML
BI Developer	Builds semantic models and reports	Power BI/DAX, SQL, Tableau
Business stakeholder	Owns a decision (marketing head, ops head). Your 'customer'	-

A **3-5 year senior analyst** is expected to do all three of these independently: write production-quality SQL, build a trustworthy dashboard/model, and use Python for statistics and light ML. This project is built to demonstrate exactly that mix.

1.4 The life-cycle of an analytics project



Always ask: 'So what? What decision will change because of this number?'

Figure 1.2 - The six-step framework used in every chapter (and in every interview answer).

1. **Clarify the business question.** 'Why is revenue growing but profit not?' is analyzable; 'analyze the data' is not. Ask who will use the answer and what decision they will make.
2. **Define the metric exactly.** Does 'revenue' include returns? Cancelled orders? Shipping? Write the definition down. Most analytics arguments are definition arguments.
3. **Find and check the data.** Where does it live? How fresh? What is the grain (one row = what)? Duplicates? Missing values? Does it reconcile to a known total?
4. **Pick the method.** Simplest thing that answers the question: a group-by before a model; a model only if prediction is needed.
5. **Analyse and validate.** Sanity-check against another source or tool. Check for confounders. Report uncertainty (confidence intervals, backtest errors).
6. **Explain and recommend.** Lead with the answer, support with 2-3 numbers, state assumptions and risks, give the next action and its estimated value.

> **INTERVIEW ANGLE**
>
> 'Walk me through how you approach a new analysis' is the most common opening question. Answer with these six steps and a one-line example for each (use the KartNova project as your example).

2. Business metrics you must know

Analysts who do not understand the business produce correct numbers that nobody uses. This chapter gives you the vocabulary of retail / e-commerce. Learn the formulas; they appear everywhere (and in many other industries under different names).

2.1 Money metrics

Metric	Plain-English meaning	Formula	KartNova example
Gross sales	Value of goods at the list (sticker) price	quantity × list price	-
Discount	Money given away from the sticker price	gross sales × discount %	avg ≈ 9-10% of gross sales
Net revenue	What the customer is invoiced after discount	gross sales − discount	₹13.4 Cr in 2025
Refunds	Money handed back when items are returned	net revenue × refund %	₹1.17 Cr in 2025
Kept revenue	Revenue after returns, the 'real' revenue	net revenue − refunds	₹12.2 Cr in 2025
COGS	Cost of Goods Sold: what the company paid for the goods	quantity × unit cost	-
Gross profit	Money left after paying for the goods	kept revenue − COGS kept	₹3.95 Cr in 2025
Gross margin %	Profit per rupee of revenue	gross profit ÷ kept revenue	32.4%

WORKED EXAMPLE

A phone has list price ₹10,000 and costs the company ₹8,000. A customer buys 1 with 10% discount.

Gross sales = 10,000. Discount = 1,000. **Net revenue = 9,000.** COGS = 8,000. **Gross profit = 1,000. Margin = 1,000 ÷ 9,000 = 11.1%.**

Notice: a 10% discount cut profit from 2,000 to 1,000, **half the profit**, because the margin was thin. That is why discounting thin-margin categories (Electronics!) is dangerous.

2.2 Order and customer metrics

Metric	Meaning	Formula / note
Orders	Number of purchase transactions	distinct order_id
Active customers	Customers who bought in the period	distinct customer_id
AOV (Average Order Value)	Typical basket size in rupees	net revenue ÷ orders (₹9,528 in 2025)
Units per order	Items per basket	units ÷ orders
Return rate	Share of sold items sent back	returned lines ÷ delivered lines (9.3%)
Cancellation rate	Share of placed orders cancelled before delivery	cancelled orders ÷ all orders (4.3%)
Repeat rate	Share of customers with 2+ orders	customers with ≥2 orders ÷ customers (62%)

2.3 Marketing metrics

Metric	Meaning	Formula	KartNova
CAC	Customer Acquisition Cost: marketing money to win one new customer	marketing spend ÷ new customers	₹778 (Referral) to ₹2,977 (Paid Search)
LTV	Customer Lifetime Value: profit a customer brings over time (here: first 12 months' gross profit)	Σ gross profit in 12 months	₹9,725 (Referral)
LTV:CAC	How many rupees of profit per rupee of acquisition cost	LTV ÷ CAC (≥3× is the common health benchmark)	Referral 12.5×, Paid Search 2.5×
Payback period	Months to earn the CAC back from profit	CAC ÷ (LTV ÷ 12)	1 month (Referral), 4.8 (Paid Search)
Churn	Customers who stop buying	1 − retention	70% did not buy in the 180 days after the cutoff
Retention	Share still buying after N months	see cohorts, Ch. 9	18.8% buy again in month 1

ANALOGY

LTV:CAC is like a bank deposit. CAC is the money you put in (to win the customer); LTV is the interest you collect. If you deposit ₹100 and earn ₹250 back, that is 2.5× and probably not worth the effort compared with a channel that returns ₹1,250 (12.5×).

2.4 Retail concepts

- **SKU (Stock Keeping Unit)**: one distinct product that can be sold. KartNova has 144.
- **Basket / cart**: all items in one order. 'Basket analysis' asks what is bought together.
- **Seasonality**: predictable repeated pattern through the year, e.g. Diwali (Oct-Nov) is peak shopping season in India.
- **COD (Cash on Delivery)**: customer pays the courier on arrival. Popular in India; higher cancellation and return risk.
- **Festive vs normal pricing**: retailers discount more in festive months, which often lowers margin even as volume rises.
- **Fiscal year (FY)**: Indian companies report April-March. FY25 = Apr-2024 to Mar-2025.
- **Pareto / 80-20 rule**: a small share of products (or customers) creates most of the revenue.
- **Channel**: how a customer found you (Organic Search, Paid Search, Social, Referral, Email, Affiliate).

COMMON MISTAKE

Never say 'revenue' without saying which one (gross, net, kept). In an interview, the first thing a good analyst does with 'calculate revenue' is ask: 'Before or after returns and discounts? Delivered orders only?'

3. Data basics: tables, keys, types and the statistics you need

3.1 What is a table?

A **table** is a grid. Each **row** is one record (one order, one customer). Each **column** is one attribute of the record (date, city, price). A **database** is a collection of related tables. A **CSV file** is just a table saved as plain text with commas between values. KartNova has 7 CSV tables.

customer_id	signup_date	city	region	acquisition_channel
C00001	2023-01-01	Mumbai	West	Organic Search
C00002	2023-01-01	Pune	West	Paid Search

Above: a tiny extract of the `customers` table. Row = one customer. This is the **grain** (what one row represents). Always state the grain of every table; most analytics bugs come from mixing grains.

3.2 Keys and relationships

- **Primary key (PK)**: a column whose value is unique per row and identifies it, e.g. `customer_id` in customers.
- **Foreign key (FK)**: a column that points to a PK in another table, e.g. `orders.customer_id` points to `customers.customer_id`.
- **One-to-many**: one customer has many orders; each order belongs to one customer. This is the most common relationship.
- **Orphan record**: a child row whose FK has no parent (an order for a customer that does not exist). We test for these.

> **ANALOGY**
>
> A school: **students** table (PK = roll number) and **marks** table (FK = roll number). Many marks rows per student. If marks contains roll number 999 but no student 999 exists, that row is an orphan: a data-quality problem.

3.3 Fact tables vs dimension tables (the star schema)

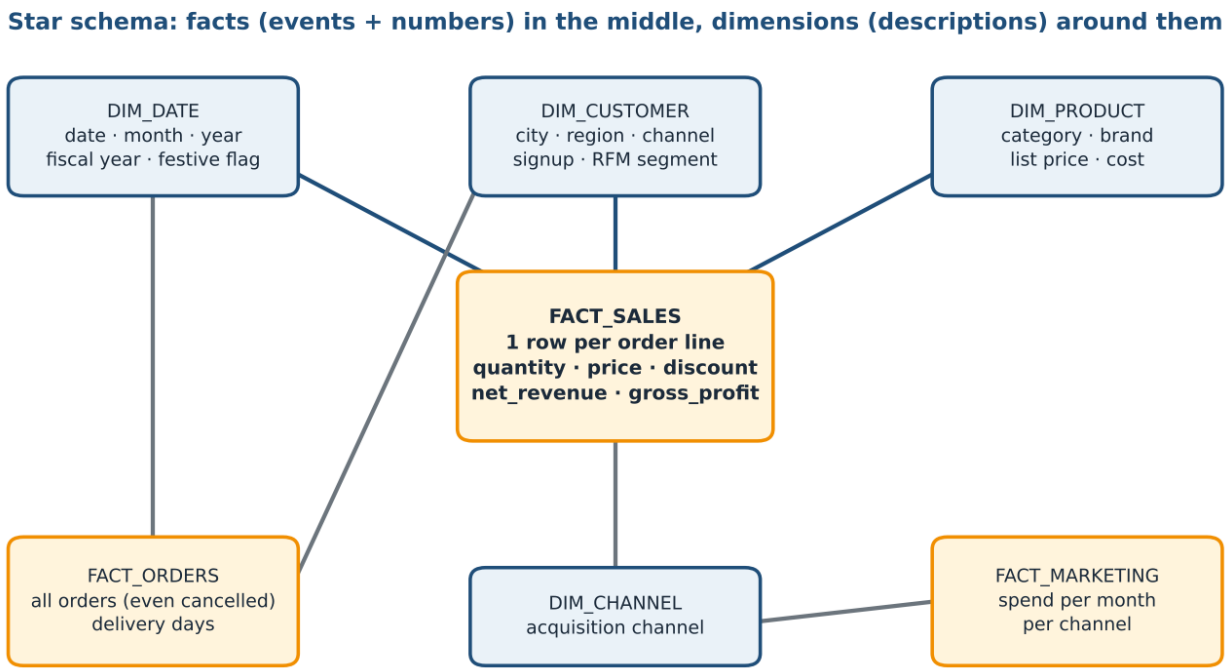


Figure 3.1 - KartNova star schema. Facts hold measurable events; dimensions describe who/what/when.

- **Fact table**: records events and the numbers about them (an order line: quantity, price, discount). Usually long (60,119 rows) and narrow.
- **Dimension table**: describes the things involved (customer, product, date). Usually short and wide.
- **Why a star?** Analysts slice facts by dimension attributes ('revenue by category by region by month'). One join from the fact to each dimension keeps queries simple, fast and unambiguous.

3.4 Data types

Type	Examples	What you can do
Numeric (continuous)	price, revenue, age	sum, average, correlate
Numeric (discrete)	quantity, order count	sum, count, distribution
Categorical (nominal)	city, category, payment method	count, group by, no ordering
Ordinal	RFM score 1-5, discount band	ordered, group by, rank
Date/time	order_date	trend, differences, seasonality
Boolean	is_returned (0/1)	average = rate (very useful!)

BEGINNER TIP

A 0/1 column's **average is a percentage**. `is_returned.mean()` = return rate. Learn this trick; it turns many questions into one line of code.

3.5 Statistics every analyst must know

Central tendency: mean vs median

The **mean** is the arithmetic average. The **median** is the middle value when sorted. Money data is almost always right-skewed (a few huge values), so the mean is pulled up by outliers while the median describes the 'typical' case.

WORKED EXAMPLE

Five customers spend: ₹1,000, ₹1,200, ₹1,500, ₹1,800, ₹100,000.

Mean = ₹21,100 (looks rich!). Median = ₹1,500 (the typical customer). In KartNova the top 10% of customers produce 48.8% of revenue, so the mean customer is not typical.

Spread: standard deviation and percentiles

Standard deviation measures how far values typically sit from the mean. **Percentiles**: the 90th percentile is the value below which 90% of data falls. A **quintile** splits data into 5 equal groups (20% each), a **decile** into 10, a **quartile** into 4. We use quintiles for RFM and deciles for churn lift.

Correlation

Correlation (−1 to +1) measures how two numbers move together. +1: perfectly together; 0: no linear relationship; −1: opposite. We used **Spearman** correlation (rank-based, robust to skew) for discount vs units and found **ρ = 0.004 ≈ 0**.

COMMON MISTAKE

Correlation is not causation. Ice-cream sales and drowning both rise in summer; ice-cream does not cause drowning (hot weather causes both). A hidden third factor is called a **confounder**; we meet one in the discount analysis (festive months raise both discounts and volume).

Distributions and the sample idea

A **population** is everything (all customers ever). A **sample** is the part we observe. Statistics lets us say how confident we can be about the population from the sample, which is the idea behind p-values and confidence intervals (Chapter 13).

4. The KartNova project: the big picture

4.1 The story

KartNova is a (simulated) Indian omnichannel e-commerce retailer selling Electronics, Fashion, Home & Kitchen, Beauty, Grocery, Sports and Books across 18 cities. We have three years of its data (Jan-2023 to Dec-2025). The leadership team wants to know: **Are we growing profitably? Which customers, products and marketing channels create value? Where do we lose money? What should we do next?** Our job is to answer with evidence.

> **IN THIS PROJECT**
>
> The dataset is **synthetic**: generated by code with realistic patterns (festive seasonality, churn, margins, returns, bundles, regional delivery differences) and deliberately injected data defects. This is a standard portfolio practice when real company data is confidential. Say so openly in interviews; the value is the method.

4.2 The data: 7 tables

Table	Grain (one row =)	Rows	Key columns
customers	one customer	9,000	customer_id, signup_date, city, region, acquisition_channel, age
products	one SKU	144	product_id, category, sub_category, brand, list_price, unit_cost
orders	one order (any status)	29,107	order_id, customer_id, order_date, delivery_date, status, payment_method, device
order_items	one product line in an order	62,700	order_item_id, order_id, product_id, quantity, unit_price, discount_pct
returns	one returned line	5,503	return_id, order_item_id, return_date, reason, refund_pct
marketing_spend	channel × month	216	month_start, channel, marketing_spend
dim_date	one calendar day	1,461	date, month, quarter, fiscal_year, is_festive_season

4.3 The pipeline

The project at a glance

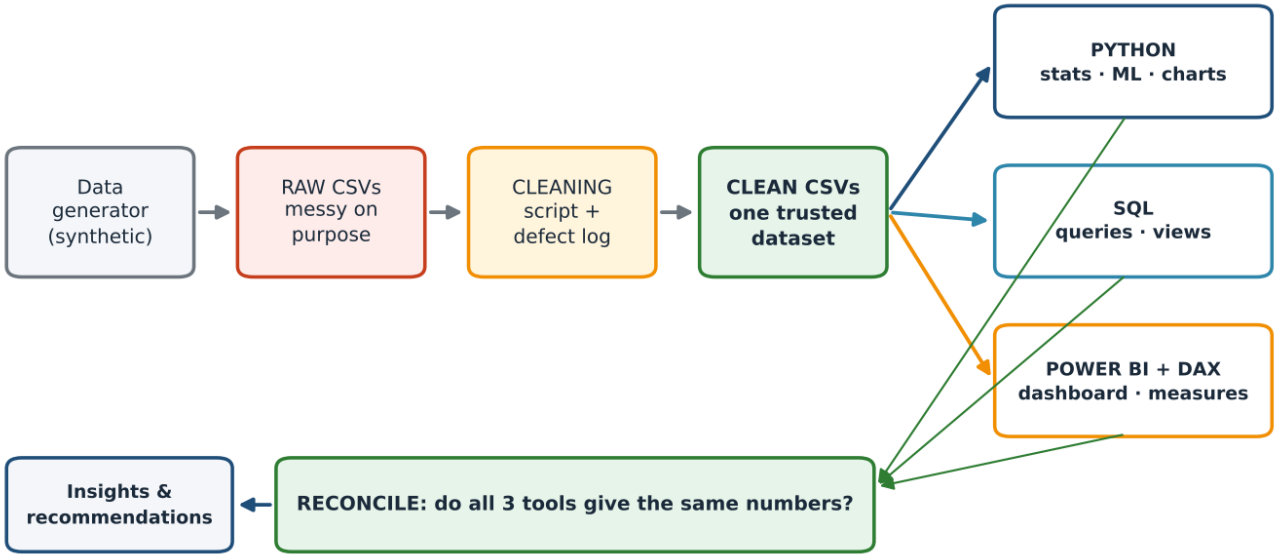


Figure 4.1 - Raw data is cleaned once; Python, SQL and Power BI then all read the same clean files. Matching totals prove correctness.

Why three tools on the same data? In real jobs different teams use different tools: engineers write SQL, analysts build dashboards, scientists use Python. A senior analyst is fluent in all three and can show the **numbers agree**. We verified, for example, 2025 net revenue = ₹133,807,961 in both Python and SQL.

4.4 Business rules (the single source of truth)

```
net_revenue = quantity × unit_price × (1 − discount%)
```

```
refund_amt = net_revenue × refund% (returned lines only)
```

```
gross_profit = net_revenue − refund_amt − COGS × (1 − refund%)
```

Only **delivered** orders count toward revenue. Cancelled orders appear only in cancellation statistics. When a line is returned, we assume returned units go back on the shelf, so their COGS is reversed in proportion to the refund. These three rules are written once and **implemented identically in Python (`build_fact`), SQL (`vw_order_lines`) and Power Query/DAX**.

> **INTERVIEW ANGLE**
>
> 'How do you make sure different reports show the same number?' Answer: define metrics once (a governed business rule or semantic layer), implement identically, and run reconciliation checks between tools. Mention the 2025 net revenue match.

4.5 Project folder map

Folder	What is inside	Read about it in
data/raw, data/clean	Messy source CSVs; cleaned CSVs shared by all tools	Ch. 6
python_analytics/	9 scripts + 15 charts + 26 result tables	Ch. 5-14
sql/	Schema, 22 analytical queries, quality checks, SQLite database	Ch. 15

Folder	What is inside	Read about it in
powerbi/	Power Query, ~80 DAX measures, DAX queries, theme, blueprint	Ch. 16
docs/	Analytics catalogue, business insights	Ch. 17

4.6 Headline results (memorise these)

Theme	Result
Growth	Net revenue ₹3.9 Cr → ₹8.9 Cr → ₹13.4 Cr (+129%, +50%) at a stable ≈32% gross margin
Portfolio	Electronics = 58% of revenue but only 22% margin; Fashion = 55% margin but 19% returns
Concentration	Top 10% customers = 49% of revenue; 25% of SKUs = 80% of revenue
Retention	18.8% buy again in month 1, 13.8% in month 6; 62% eventually place a 2nd order
Churn model	ROC-AUC 0.83; top 4 risk deciles capture 54% of churners
Marketing	LTV:CAC Referral 12.5× vs Paid Search 2.5×
Pricing	Discount depth does not raise units (ρ=0.004); a 10% cap ≈ +₹83.9 L profit
Operations	COD cancels 8.9% vs 3.3%; North-East: 54% of orders take > 7 days

5. Python for analysts: pandas in one chapter

Python is a programming language; **pandas** is its table-handling library (think 'Excel with superpowers and a memory'). The central object is the **DataFrame**: a table with labelled columns. If you understand the 10 operations below you can read every Python script in this project.

> **ANALOGY**
>
> Excel is like cooking by hand: great for one meal. pandas is a commercial kitchen: the same recipe (code) can be run on 10 rows or 10 million rows, repeated tomorrow, and reviewed by a colleague.

5.1 The ten operations

1) Load a table and look at it

```
import pandas as pd
orders = pd.read_csv("orders.csv", parse_dates=["order_date"])   # parse_dates -> real
    dates, not text
orders.head()        # first 5 rows            orders.shape   # (rows, columns)
orders.info()        # column types, nulls     orders.describe()   # count, mean, min,
    max...
```

2) Select columns and filter rows

```
orders[["order_id", "order_status"]]                    # pick columns
delivered = orders[orders.order_status == "Delivered"]    # filter rows (a boolean mask)
big = orders[(orders.shipping_fee == 0) & (orders.device == "Desktop")]   # & = AND, | =
     OR
```

3) Create new columns

```
items["gross_sales"] = items.quantity * items.unit_price
items["net_revenue"] = items.gross_sales * (1 - items.discount_pct / 100)   #
    vectorised: all rows at once
```

4) Group-by: split → apply → combine (the most important idea)

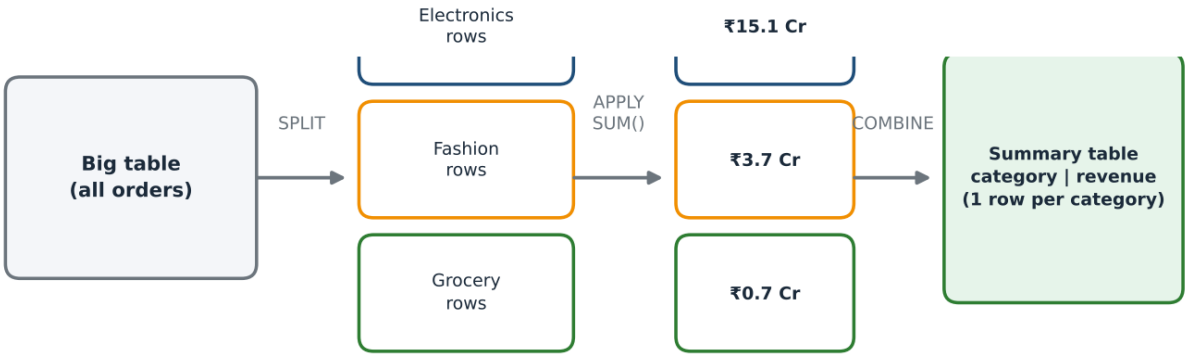


Figure 5.1 - groupby splits rows into groups, applies a function to each, and combines the answers.

```
cat = f.groupby("category").agg(revenue=("net_revenue", "sum"),
                                return_rate=("is_returned", "mean"),
                                orders=("order_id", "nunique"))
```

Real line from 02_eda_and_kpis.py. `nunique` *counts distinct values;* `mean` *of a 0/1 column is a rate.*

5) Merge (join) tables

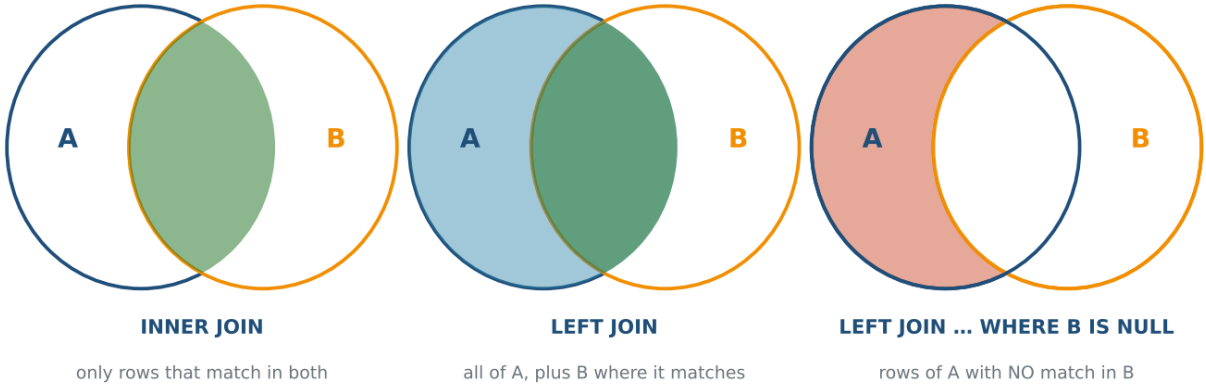


Figure 5.2 - join types. `how='inner'` *keeps matches only;* `how='left'` *keeps all rows from the left table.*

```
f = items.merge(orders, on="order_id")                    # attach order info to each
    line
f = f.merge(returns, on="order_item_id", how="left")      # left: keep lines that
    were NOT returned
```

COMMON MISTAKE

Join explosion. If the right table has more than one row per key, a join multiplies rows and silently inflates totals. After every merge print `len()` before/after, and check that grain is unchanged. KartNova's returns table has exactly one row per returned line (`UNIQUE` on `order_item_id`) to avoid this.

6) Dates

```
orders["order_month"] = orders.order_date.dt.to_period("M").dt.to_timestamp()   # month
    bucket
orders["delivery_days"] = (orders.delivery_date - orders.order_date).dt.days
```

7) Missing values

```
df.isna().sum()                    # nulls per column
df["age"] = df.age.fillna(df.age.median())     # fill with median
df = df.dropna(subset=["order_date"])          # or drop rows
```

8) Duplicates

```
orders.duplicated("order_id").sum()       # how many repeats?
orders = orders.drop_duplicates("order_id")
```

9) Pivot / unstack: long → wide

```
cohort = o.groupby(["cohort", "age_m"]).customer_id.nunique().unstack(fill_value=0)   #
    rows=cohort, cols=month index
```

10) Buckets and ranks

```
rfm["R"] = pd.qcut(rfm.recency, 5, labels=[5, 4, 3, 2, 1])   # quintiles: 5 equal-sized
    groups
pd.cut(df.age, [17, 24, 34, 44, 54, 120], labels=["18-24", "25-34", "35-44", "45-54",
    "55+"])   # fixed-width bins
```

5.2 The project's core helper: `build_fact()`

Almost every analysis starts with one function that builds the **analysis-ready table**: order lines + order info + product info + customer info + return info + money columns. Building it once and reusing it guarantees every script uses the same business rules.

```
def build_fact(d=None, delivered_only=True):
    f = (d["order_items"].merge(d["orders"], on="order_id")
          .merge(d["products"][[...]], on="product_id")
          .merge(d["customers"][[...]], on="customer_id")
          .merge(d["returns"][[...]], on="order_item_id", how="left"))
    if delivered_only: f = f[f.order_status == "Delivered"].copy()
    f["gross_sales"]  = f.quantity * f.unit_price
    f["net_revenue"]  = f.gross_sales * (1 - f.discount_pct / 100)
    f["is_returned"]  = f.return_date.notna()
    f["refund_amt"]   = np.where(f.is_returned, f.net_revenue * f.refund_pct.fillna(0),
        0.0)
    f["cogs"]         = f.quantity * f.unit_cost
    f["kept_revenue"] = f.net_revenue - f.refund_amt
    restock = np.where(f.is_returned, f.refund_pct.fillna(0), 0.0)
    f["gross_profit"] = f.kept_revenue - f.cogs * (1 - restock)
    return f
```

Simplified from python_analytics/utils.py.

INTERVIEW ANGLE

'What does this function return and what is its grain?' → One row per delivered order line (60,119 rows) with all attributes and money columns. 'Why one function?' → single source of truth; changing a rule in one place updates every analysis.

BEGINNER TIP

Try it yourself: open a Python session, run `f = build_fact()`, then answer: (a) total net revenue, (b) revenue by region, (c) return rate by category. Each is one `groupby` line. Compare your answers with Chapter 7.

6. Data cleaning: trusting your data

Rule #1 of analytics: garbage in, garbage out. Real data always has problems. A good analyst finds them **before** analysing, fixes them with explicit rules, and **documents what changed**. The cleaning script (`01_data_cleaning.py`) does exactly that and writes a log so a reviewer can audit every decision.

6.1 What we found and what we did

table	issue	rows	action
customers	duplicate customer_id	0	drop
customers	inconsistent city casing / whitespace	450	strip + title-case
customers	impossible age (<18 or >80)	8	set NaN
customers	missing age (incl. invalid)	368	impute median by city_tier
customers	missing gender	180	label 'Unknown'
orders	duplicate order rows	291	drop
orders	delivery_date earlier than order_date	25	re-impute order_date + 5d (≈ median lead time)
orders	missing payment_method	728	label 'Unknown'
orders	orphan customer_id (FK check)	0	none found
order_items	duplicate line items	30	drop
order_items	non-positive quantity	12	set to 1
order_items	orphan order_id (FK check)	0	none found
order_items	orphan product_id (FK check)	0	none found
returns	orphan order_item_id (FK check)	0	none found
dim_date	generated calendar 2023-2026 (+fiscal, festive flag)	1461	created

6.2 Why each fix was chosen

Defect	Why it matters	Why this fix (and the alternative)
Duplicate rows (291 orders, 30 lines)	Double-counts revenue and orders	Drop by primary key. Alternative: investigate source system. Always count before/after.
Inconsistent city text ('MUMBAI', ' mumbai ')	Same city appears as several groups in a group-by	strip() + title-case. Standardise text before grouping.
Impossible ages (150, 0, −5, 999)	Distorts averages and age bands	Set to NaN then impute. Never silently keep impossible values.
Missing age (368 incl. invalid)	Cannot band or model without it	Median **by city tier** (median is outlier-resistant; group-wise keeps some signal). Alternative: leave missing and add a flag.
Missing payment method (728)	Hides a real category	Label 'Unknown' instead of guessing; 'Unknown' itself became analyzable (4.7% cancels).
Delivery date before order date (25)	Negative delivery days poison averages	Re-impute order_date + 5 days (≈ median lead time). Alternative: set null and exclude.
Quantity ≤ 0 (12)	Negative revenue	Set to 1 (minimum valid). Alternative: drop the line. Document the choice.
Orphan keys (0 found)	Broken joins drop rows silently	Test anyway; 'none found' is a valid and useful result.

KEY IDEA

Cleaning is **decisions**, not typing. For each defect an interviewer wants to hear: how you detected it, how many rows, the fix, **the alternative you rejected and why**, and the impact on results.

6.3 Code pattern: detect → count → fix → log

```
bad_age = ((cust.age < 18) | (cust.age > 80)).sum()              # detect + count
cust.loc[(cust.age < 18) | (cust.age > 80), "age"] = np.nan       # fix step 1
med = cust.groupby("city_tier").age.transform("median")           # group-wise median
cust["age"] = cust.age.fillna(med).round().astype(int)            # fix step 2
note("customers", "impossible age", bad_age, "set NaN -> impute median by city_tier")
    # log
```

`transform('median')` returns a column the same length as the table, filled with each row's **group's** median, which is why it can be passed straight into `fillna`.

6.4 Validation checks after cleaning

- **Row counts** before/after each step (nothing vanished unexpectedly).
- **Uniqueness** of primary keys.
- **Referential integrity**: every order has a customer; every line has an order and product; every return has a line.
- **Business rules**: delivery ≥ order date; discount between 0 and 100; list price ≥ unit cost; cancelled orders have no returns.
- **Reconciliation**: totals match another source (Python = SQL here).

INTERVIEW ANGLE

'How do you handle missing data?' → First understand **why** it is missing (random vs systematic). Options: drop, impute (mean/median/mode/group-wise/model), flag, or leave as its own category. Choose by how much is missing and how the field is used; always document and test sensitivity.

7. Executive KPIs and exploratory analysis (EDA)

EDA (Exploratory Data Analysis) means looking at the data from many angles to learn what is going on before deeper modelling. Output of this stage: the KPI scorecard leaders read, and a list of 'interesting facts' that become the questions for later chapters.

7.1 The KPI scorecard

Year	Net revenue ₹	GM %	orders	Customers	AOV ₹	Return %	Cancel %	YoY %
2023	38,809,309	31.69	4,022	1,956	9,649	9.5	4.42	-
2024	88,935,118	32.42	9,805	4,204	9,070	8.85	4.08	129
2025	133,807,961	32.38	14,044	5,573	9,528	9.27	4.31	50

How to read it: revenue more than doubled in 2024 (+129%) because the platform was young and customers were accumulating; 2025 grew 50% on a bigger base. **Gross margin is stable** (≈32%), so growth is not being bought by margin erosion. AOV is flat (≈₹9-9.6k), so growth comes from **more orders and customers**, not bigger baskets.

> **WORKED EXAMPLE**
>
> YoY growth = (this year ÷ last year) − 1. 2025: 133.8 ÷ 88.9 − 1 = 0.5046 → **+50.5%**.
>
> A growth rate always falls as the base grows; 50% on ₹8.9 Cr is harder than 129% on ₹3.9 Cr. Do not read falling growth % as failure without checking absolute rupees added (₹4.5 Cr vs ₹5.0 Cr).

7.2 Trend and seasonality

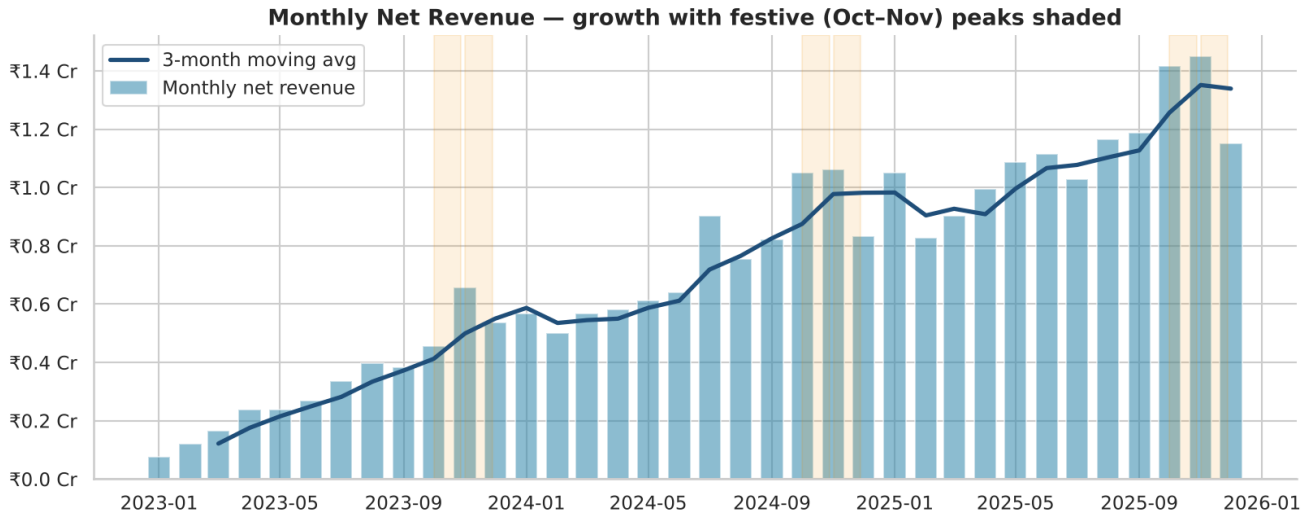


Figure 7.1 - Monthly net revenue (bars), 3-month moving average (line). Orange bands = Oct-Nov festive season.

A **moving average** replaces each month with the average of the last 3, smoothing noise to reveal direction. **Seasonality** is a repeating pattern: Oct-Nov peaks every year (Diwali). Seasonality index = month revenue ÷ average month. ≈1.4 for October and ≈1.6 for November means those months run 40-60% above an average month.

COMMON MISTAKE

Our seasonality index inherits the **growth trend**: a growing business shows late-year months above average even without a festival. A proper decomposition (STL / ratio-to-moving-average) removes trend first. In an interview, volunteering this caveat shows seniority.

7.3 Category economics: revenue ≠ profit

category	Revenue share %	Gross margin %	Return rate %	Avg discount %
Electronics	57.9	21.7	8.1	7.1
Fashion	14.2	55.4	19.4	12.3
Home & Kitchen	10.6	38.3	5.7	12.3
Sports & Fitness	6.4	46.3	9.4	12.6
Beauty & Personal Care	5.8	62.1	6.6	12.6
Grocery & Gourmet	2.9	26.8	2.2	8.4
Books & Stationery	2.4	35.8	3.1	10.7

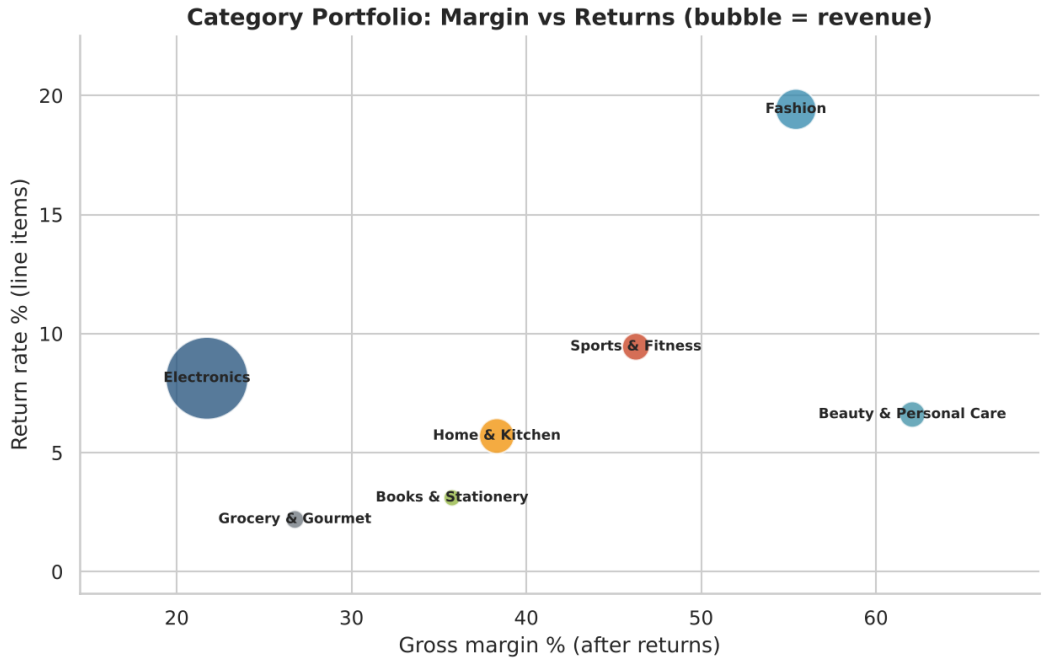


Figure 7.2 - Bubble chart: x = margin, y = return rate, size = revenue. Top-right = high margin but returns leak it.

Electronics is the engine of revenue (58%) but the weakest in margin (22%). **Fashion** has a 55% margin but a 19% return rate, so a fifth of its sales come back. **Beauty** is the healthiest quadrant: high margin and low returns. Each category needs a different playbook: protect Electronics margin, fix Fashion returns, grow Beauty.

7.4 Pareto (80/20) analysis

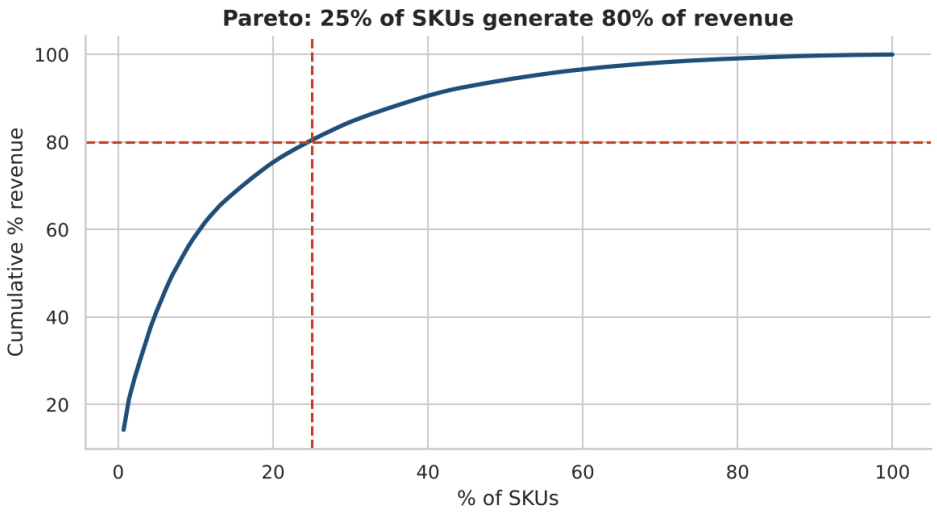


Figure 7.3 - Cumulative revenue vs share of SKUs. About a quarter of SKUs create 80% of revenue.

Sort products from best to worst, add up cumulative revenue share. Where the curve crosses 80% tells you the concentration. **ABC classification** uses it for inventory: **A** = top products making the first 70% of revenue (never stock out), **B** = next 20%, **C** = the tail (candidates to delist or sell on demand).

7.5 Operations: regions and payment

region	customers	Avg delivery days	Return %
West	2865	4.9	9.0
South	2541	5.2	9.4
North	2123	5.4	9.1
East	907	6.3	8.8
Central	276	5.7	9.0
North-East	142	7.7	10.2

North-East takes 7.7 days on average (West 4.9) and 54% of its orders exceed 7 days. COD orders cancel at 8.85% vs ~3% for digital payments (tested formally in Chapter 13).

INTERVIEW ANGLE

'Revenue grew 50% but the CFO says profit did not. How do you investigate?' Decompose: volume vs price vs mix vs discount vs returns vs cost. Check margin by category (mix shift toward low-margin Electronics?), discount trend, return rate, and COGS changes. Here, margin was flat, so growth is healthy; but 58% of revenue sits in the 22%-margin category, so mix is the risk to watch.

'Which KPIs would you put on a CEO dashboard?' 5-7 only: net revenue + YoY, gross margin %, orders, AOV, active customers, return rate, with a trend and a drill-down by category/region.

8. Customer segmentation: RFM and K-Means

8.1 Why segment customers?

Treating 9,000 customers identically wastes money: a loyal buyer does not need a discount; a lapsed one might. **Segmentation** groups similar customers so each group gets the right action. The classic behavioural method is **RFM**.

> **ANALOGY**
>
> A restaurant owner knows three kinds of diners: regulars (come weekly, spend well: greet by name), tourists (come once: try to make them regulars), and old regulars who stopped coming (send a 'we miss you' offer). RFM is the owner's instinct turned into numbers.

8.2 RFM defined

Letter	Question	How measured here	Good is...
Recency	How recently did they buy?	days from last order to 31-Dec-2025	low (recent)
Frequency	How often do they buy?	number of distinct orders	high
Monetary	How much do they spend?	kept revenue (after returns)	high

8.3 Scoring with quintiles

Rank customers on each measure and cut into **5 equal groups** (quintiles). Score 5 = best 20%, 1 = worst 20%. For recency, fewer days is better, so the labels are reversed (5 = most recent).

> **WORKED EXAMPLE**
>
> Eight customers' recency in days: 5, 12, 30, 45, 90, 150, 300, 600. Two customers per quintile.
>
> Days 5 and 12 → R=5; 30 and 45 → R=4; 90 and 150 → R=3 ... wait, that gives only three groups: with 8 customers and 5 groups it is uneven, which is why we rank first (`rank(method='first')`) so ties and uneven sizes still split cleanly.

```
rfm["R"] = pd.qcut(rfm.recency, 5, labels=[5, 4, 3, 2, 1]).astype(int)          # low
    days = score 5
rfm["F"] = pd.qcut(rfm.frequency.rank(method="first"), 5,
    labels=[1,2,3,4,5]).astype(int)
rfm["M"] = pd.qcut(rfm.monetary.rank(method="first"),  5,
    labels=[1,2,3,4,5]).astype(int)
```

`rank(method='first')` *breaks ties so qcut never fails with duplicate edges (many customers have exactly 1 order).*

8.4 From scores to named segments

Segment	Rule (R, F, M scores)	Meaning	Action
Champions	R≥4, F≥4, M≥4	Recent, frequent, big spenders	Reward, early access, referral ask
Loyal Customers	R≥3, F≥4	Buy often	Upsell, loyalty tier
Potential Loyalists	R≥3, F≥3	Doing well, not yet loyal	Nudge to next purchase
New / Promising	R≥4, F≤2	Recent, few orders	Onboarding, 2nd-order incentive

Segment	Rule (R, F, M scores)	Meaning	Action
Can't Lose Them	R≤2, F≥4, M≥4	Were great, gone quiet	Personal win-back (highest value!)
At Risk	R≤2, F≥3	Slipping away	Win-back campaign
Lost	R≤2, F≤2, M≤2	Gone, low value	Stop discounting; low-cost reactivation only
Needs Attention	everyone else	Middle	Test offers

8.5 What we found

segment	customers	% custs	% revenue	Avg days since last	Avg orders
Champions	1,476	16.7	42.9	52	7.3
Can't Lose Them	645	7.3	15.7	526	5.4
Loyal Customers	1,114	12.6	14.1	161	4.7
Needs Attention	1,347	15.2	8.9	437	1.1
Potential Loyalists	1,068	12.1	7.6	126	2.0
At Risk	1,009	11.4	6.3	535	2.5
New / Promising	898	10.1	3.4	80	1.0
Lost	1,297	14.7	1.3	655	1.1

Champions are 17% of customers but 43% of revenue. 'Can't Lose Them' are 7% of customers with 16% of revenue and have been silent for about 17 months: the single most valuable win-back audience. 'Lost' customers (15% of the base) are just 1.3% of revenue, so spending discounts on them is poor value.

8.6 K-Means clustering: letting the data find groups

RFM rules are chosen by humans. **K-Means** is an **unsupervised** algorithm (no right answers given) that discovers groups on its own. We use it as a **cross-check**: if data-driven clusters resemble our named segments, our segments are sensible.

How K-Means works (4 steps)

1. Choose **k** (number of clusters). Place k random 'centroids' (cluster centres).
2. **Assign** every customer to the nearest centroid (distance in R-F-M space).
3. **Move** each centroid to the average position of its customers.
4. Repeat steps 2-3 until nothing changes.

> **ANALOGY**
>
> Put k pins on a map of your city's customers. Each customer joins the nearest pin. Move each pin to the middle of its customers. Repeat. Pins settle at natural customer clusters.

Two preparation steps you must explain

- **Log transform** (`log1p`): money and frequency are very skewed; log compresses the long tail so a few giant customers do not dominate.
- **Standardise** (`StandardScaler`): rescale each column to mean 0, std 1, so ₹ amounts (thousands) do not drown out recency (days) or frequency (single digits). K-Means uses distance, so scale matters.

Choosing k: the silhouette score

The **silhouette score** (−1 to +1) measures how well each point fits its own cluster versus the next nearest one. Higher = tighter, better-separated clusters. We tested k = 3 to 8: scores 0.293, **0.307**, 0.302, 0.290, 0.285, 0.284, so **k = 4** wins (we started at 3 because k = 2 is a trivial 'active vs lapsed' split).

cluster	Customers	Avg recency (days)	Avg frequency	Avg monetary ₹
0	1,392	63.2	8.4	77635.1
1	1,796	62.9	2.0	12928.0
2	2,940	479.5	1.2	3310.5
3	2,726	428.6	3.4	35990.6

Reading the clusters: cluster 0 = recent, frequent, high-value (≈ Champions); cluster 1 = recent, mid-value, newer; cluster 2 = long-lapsed one-timers (≈ Lost/Needs Attention); cluster 3 = lapsed but previously decent buyers (≈ At Risk/Can't Lose). The overlap with the rule-based segments is the validation.

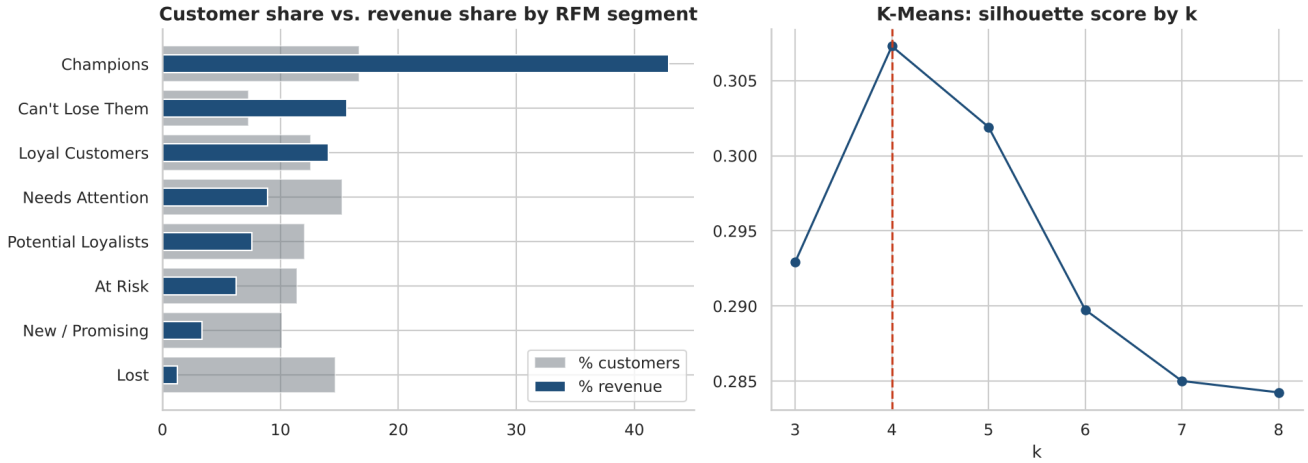


Figure 8.1 - Left: customer share vs revenue share per RFM segment. Right: silhouette score by k.

COMMON MISTAKE

K-Means assumes roughly round, similarly sized clusters and needs numeric scaled inputs; results depend on the random start (use `n_init` and `random_state`). Silhouette ≈ 0.3 means 'reasonable structure', not crisp clusters; honest reporting beats overselling.

INTERVIEW ANGLE

'RFM vs K-Means: which would you use?' RFM for communication and activation (marketers understand 'Champions'); K-Means to validate or when you want more dimensions. Often both.

'How do you decide the number of clusters?' Silhouette/elbow for statistics, then business usefulness: can the team run a different campaign for each?

'What if frequency is mostly 1?' Use `rank(method='first')` before quintiles, or custom cut-offs.

9. Cohort retention analysis

Total revenue can grow while customers quietly leave, if new customers replace them (a **leaky bucket**). **Cohort analysis** checks the bucket. A **cohort** is a group of customers who started in the same period (here: month of first purchase). We track what share of each cohort keeps buying in later months.

ANALOGY

A school tracks each year's admissions batch separately: of the 100 students who joined in 2020, how many are still enrolled after 1, 2, 3 years? Comparing batches shows whether the school is getting better at keeping students.

9.1 How to read a cohort table

Cohort	Size	M0	M1	M2	M3
Jan	100	100%	20%	17%	15%
Feb	120	100%	22%	18%	
Mar	150	100%	21%		
Apr	90	100%			

blank, NOT 0%: this month hasn't happened yet (right-censoring)

Read a row left→right: of the people who first bought in Jan, 20% bought again the next month (M1), 17% in M2...

Figure 9.1 - A mini cohort table. Rows = first-purchase month; columns = months since first purchase.

- **M0** is always 100% (everyone bought in their first month by definition).
- **M1** = share who bought again in the next month, **M2** the month after, and so on.
- **Blank, not 0%:** a cohort that started in April cannot have a M3 yet. Showing 0% would wrongly say everyone left. This is called **right-censoring**.

COMMON MISTAKE

Showing 0 instead of blank for unobserved cells is the #1 cohort mistake. It makes recent cohorts look terrible. We explicitly blank cells whose calendar month is beyond the last data month.

9.2 How the code builds it

1. Get one row per order: customer, order month.
2. Find each customer's **first order month** = their cohort.
3. For each order compute `age_m` = months since cohort start.
4. Count **distinct customers** active at each (cohort, age_m).
5. Divide by cohort size (the age 0 count) to get retention %.

```
o["order_m"] = o.order_date.dt.to_period("M")
first = o.groupby("customer_id").order_m.min().rename("cohort")       # step 2
o = o.join(first, on="customer_id"); o["age_m"] = (o.order_m - o.cohort).apply(lambda x:
     x.n)
cohort = o.groupby(["cohort", "age_m"]).customer_id.nunique().unstack(fill_value=0)   #
    step 4
retention = cohort.div(cohort[0], axis=0) * 100                        # step 5
```

9.3 What we found

Month after first order	Retention (size-weighted, all cohorts)
1	18.8%
3	16.3%
6	13.8%

About one in five customers buys again in the next month and about one in seven is still buying six months later: the curve **flattens**, which is healthy (a stable loyal core). Yet 62% of customers eventually place a second order after an average of **95 days**, meaning many come back *later* than the next month: a clue for timing re-engagement messages around day 60-90.

9.4 Retention by acquisition channel

Month	Affiliate	Email Marketing	Organic Search	Paid Search	Referral	Social Media
0	100.0	100.0	100.0	100.0	100.0	100.0
1	17.6	17.3	20.5	17.6	19.3	17.4
2	17.7	18.9	18.9	15.6	17.6	14.5
3	14.7	14.9	17.4	15.3	17.5	15.8
4	15.4	17.3	18.3	12.9	19.4	15.3
5	13.8	15.8	16.9	13.3	17.7	11.2
6	12.5	17.3	15.6	12.1	17.6	11.3

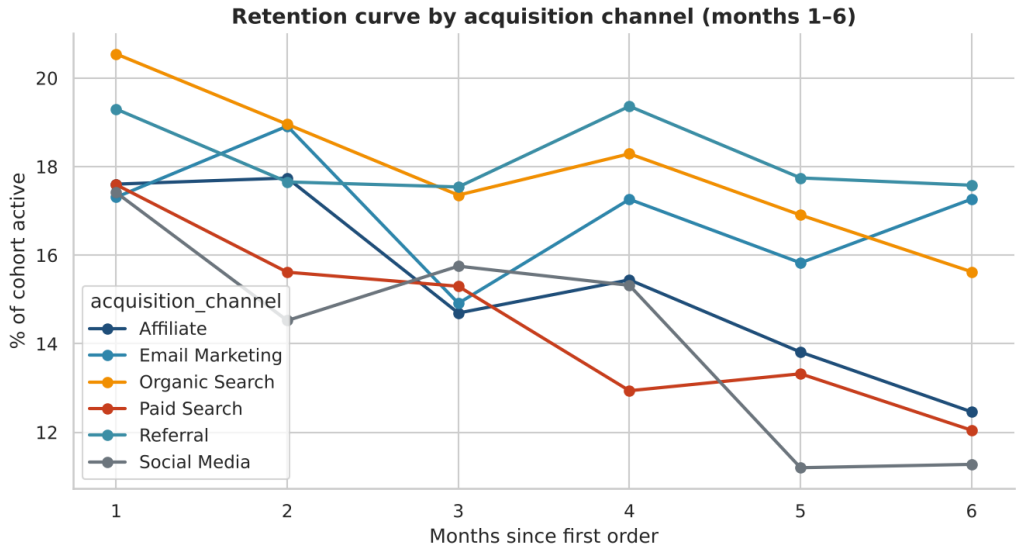


Figure 9.2 - Retention curves by channel (cohorts with ≥ 6 months of history for fairness).

Referral and Organic Search retain best (≈17-18% at month 6 for Referral vs ≈11% for Social Media and ≈12% for Paid Search). Cheap-to-acquire social customers also leave sooner; this connects directly to the LTV:CAC result in Chapter 11.

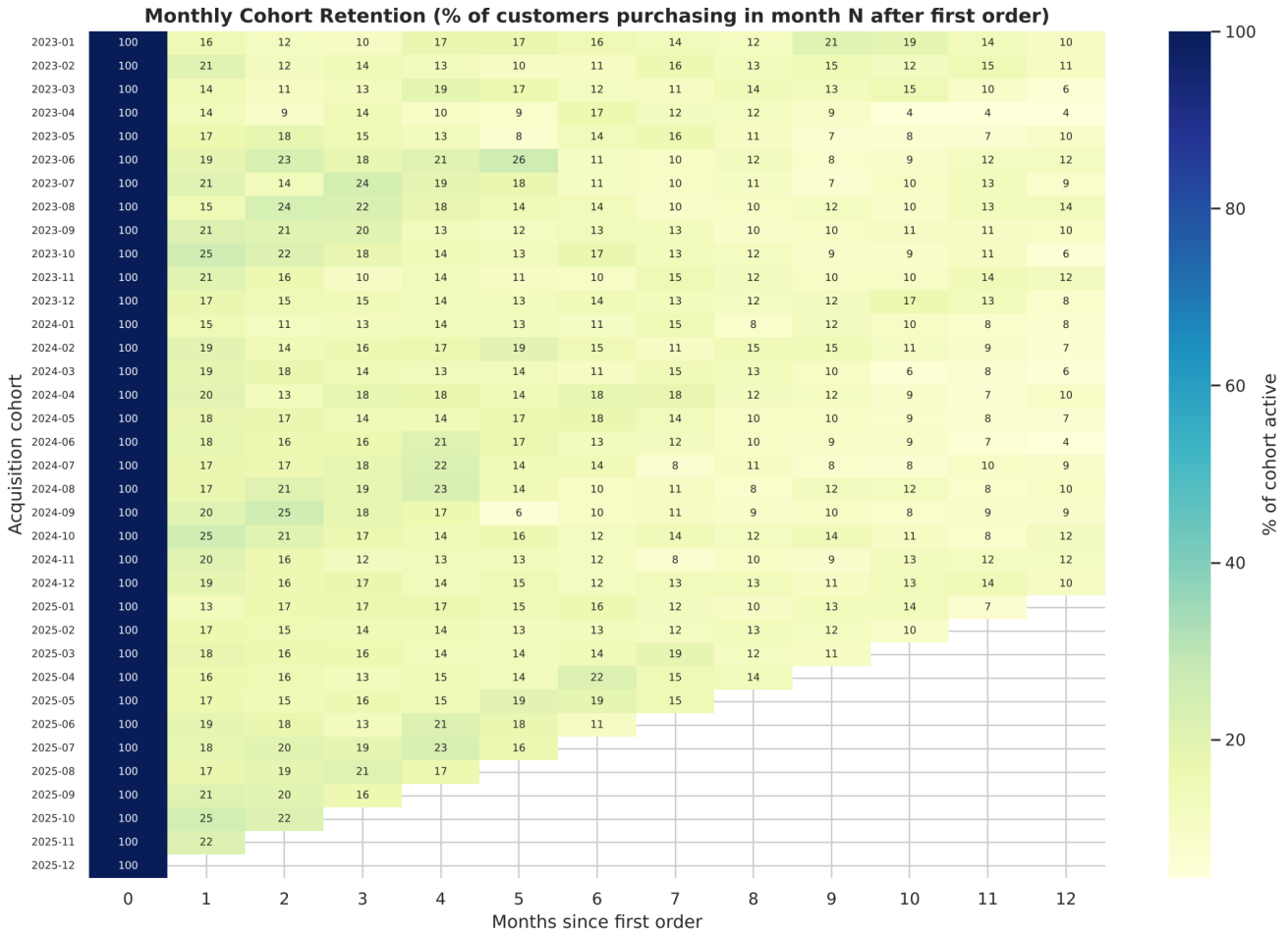


Figure 9.3 - Retention heatmap for every cohort (darker = higher). Staircase of blanks at the right = right-censoring.

INTERVIEW ANGLE

'How do you measure retention?' Define 'active' (a purchase), define cohort (first purchase month), count distinct customers per month since start, divide by cohort size; blank unobserved periods.

'Cohort vs aggregate churn: why cohorts?' Aggregate rates mix old and new customers and hide whether recent cohorts are better; cohorts separate the effect of age from the effect of time.

'Retention is 14% at month 6 - good or bad?' Depends on purchase frequency of the business. For occasional-purchase e-commerce it is plausible; benchmark against own history and category peers, and look at trend across cohorts.

10. Churn prediction: machine learning from zero

10.1 The business problem

Churn means a customer stops buying. Winning a new customer costs 5-7× more than keeping one, so if we could predict who is about to leave we could spend the retention budget (vouchers, calls) **only on the right people**. We define: a customer **churned** if they placed **no delivered order in the 180 days after 30-Jun-2025**.

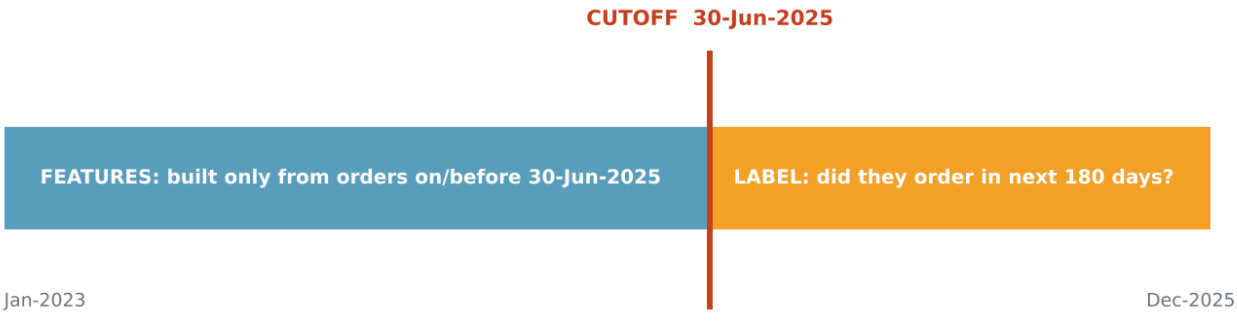
10.2 Machine learning in plain English

Supervised machine learning learns a pattern from examples where the answer is known, then applies it to new cases. Each customer is one example with **features** (inputs describing the customer) and a **label** (the answer we want to predict, here churned yes/no).

> **ANALOGY**
>
> A bank officer who has seen 10,000 past loan applicants (features: income, job, history) and whether each repaid (label) develops a feel for who is risky. A model is that experience written as a formula, learned from data instead of from intuition.

Term	Meaning	In this project
Features (X)	Input columns	recency, frequency, monetary, discount habit, return rate, channel, region...
Label (y)	What we predict	1 = churned in next 180 days, 0 = bought again
Training set	Data the model learns from (75%)	≈ 5,300 customers
Test set	Held-out data to judge honestly (25%)	≈ 1,770 customers never seen in training
Overfitting	Memorising training quirks; great on train, poor on new data	Why we evaluate only on the test set

10.3 The most important rule: avoid data leakage



Leakage = letting information from AFTER the cutoff sneak into the features (the model would 'cheat').

Figure 10.1 - Features may only use information available at the cutoff; the label comes from afterwards.

Leakage is when information from the future sneaks into the features, making the model look brilliant in testing and useless in real life. Our design is **temporal**: every feature is computed from orders on or before 30-Jun-2025; the label is observed only in the following 180 days. If we had included 'orders in the last 30 days of December' as a feature, the model would trivially 'predict' churn from the answer itself.

> **COMMON MISTAKE**
>
> Subtle leakage examples: using a customer's total lifetime spend (includes future), scaling the full dataset before splitting, or random-splitting time-ordered data when the model will be used on later periods. Always ask: 'Would I have this value at prediction time?'

10.4 The features we built

Group	Features	Why they might predict churn
RFM	recency_days, frequency, monetary, avg_order_value, tenure_days	Recent, frequent buyers keep buying
Rhythm	avg_days_between_orders, recency_to_gap_ratio, orders_last_90d	A customer overdue relative to their own pattern is at risk
Experience	return_rate, avg_delivery_days, cod_share, avg_discount_pct	Bad experiences and deal-hunting behaviour
Preference	n_categories, electronics_share, app_share	Breadth of engagement
Profile	acquisition_channel, region, age (one-hot encoded)	Some channels bring stickier customers

One-hot encoding turns a category into 0/1 columns (`acquisition_channel_Referral` = 1 if Referral else 0) because models need numbers.

10.5 The two models

Logistic regression (the baseline)

Despite the name it classifies. It computes a weighted sum of features and squeezes it into a probability between 0 and 1 using the **sigmoid** function:

```
probability(churn) = 1 / (1 + e^-(b0 + b1·recency + b2·frequency + ...))
```

Training finds the weights (b) that best match history. It is simple, fast and **explainable**: each weight says how a feature pushes risk up or down.

Gradient boosting (the challenger)

Builds hundreds of small decision trees one after another; each new tree focuses on the mistakes of the previous ones. It captures non-linear patterns and interactions but is a 'black box' and can overfit.

> **KEY IDEA**
>
> **Always start with a simple baseline.** A complex model must **beat** the baseline by a worthwhile margin to justify its cost and opacity. Here it did not, which is itself a result worth reporting.

10.6 How to judge a classifier

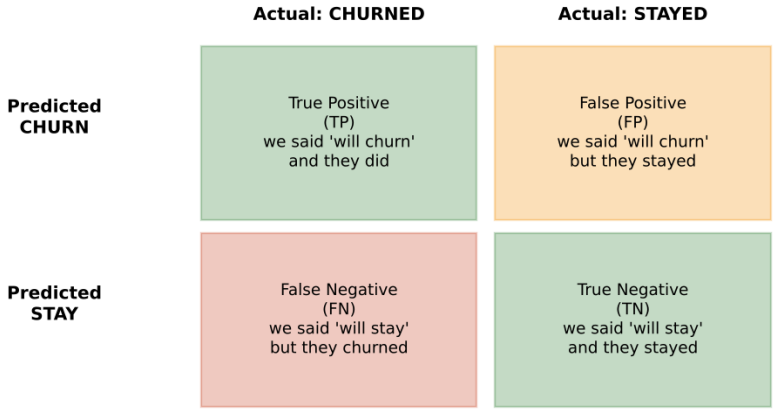


Figure 10.2 - The confusion matrix: four outcomes for a yes/no prediction.

Metric	Formula / meaning	Watch out
Accuracy	(TP+TN) ÷ all	Misleading: 70% of customers churn, so 'predict churn for everyone' scores 70%
Precision	TP ÷ (TP+FP): of those we flag, how many truly churn	Matters when each action costs money
Recall	TP ÷ (TP+FN): of all churners, how many we catch	Matters when missing a churner is costly
ROC-AUC	Probability a random churner is scored higher than a random non-churner (0.5 = coin flip, 1 = perfect)	Threshold-free ranking quality
PR-AUC	Area under precision-recall curve	Better when classes are imbalanced
Brier score	Mean squared error of the probabilities (lower better)	Checks probability quality, not just ranking

WORKED EXAMPLE

ROC-AUC intuition: pick one customer who churned and one who did not, at random. AUC = the chance the model gave the churner the higher risk score. 0.825 means 82.5% of such pairs are ordered correctly.

10.7 Results

model	ROC-AUC	PR-AUC	Brier
Logistic Regression (baseline)	0.8254	0.9218	0.1508
Gradient Boosting	0.8124	0.9159	0.1564

Logistic regression (0.825) slightly beat gradient boosting (0.812). Why? Features are dominated by one signal, **recency**, which is roughly monotonic with churn, so the extra flexibility of boosting adds little but variance. We therefore choose the simpler, explainable model.

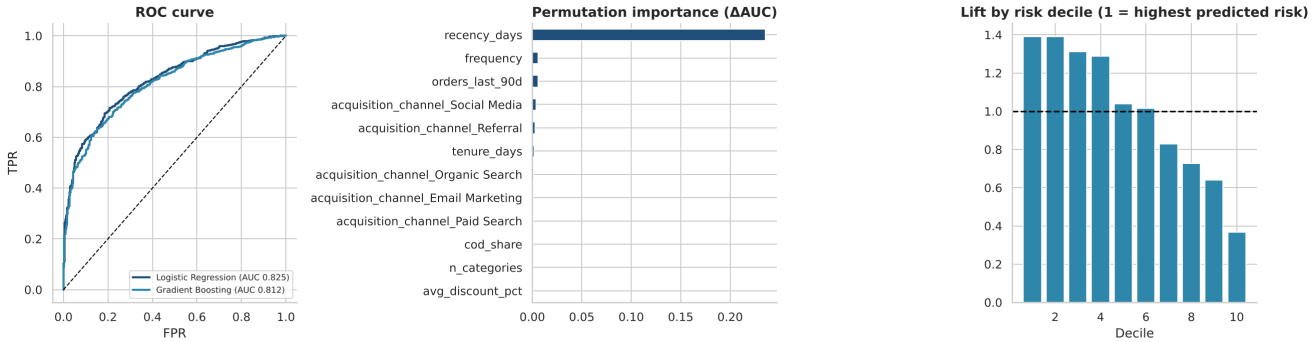


Figure 10.3 - ROC curves (left), permutation importance (middle), lift by risk decile (right).

Permutation importance: which features matter?

Shuffle one feature's values at random and see how much the model's AUC drops. A big drop = the model relied on it. **recency_days: ΔAUC = 0.236**, nearly everything else < 0.006. A customer's silence is by far the best warning sign.

Decile lift: turning a model into a campaign plan

decile	customers	Actual churn rate	Lift	Cum. % of churners captured
1	177	98%	1.39	13.9
2	177	98%	1.39	27.9
3	177	93%	1.31	41.0
4	176	91%	1.29	53.8
5	177	73%	1.04	64.2
6	177	72%	1.02	74.4
7	176	59%	0.83	82.6
8	177	51%	0.73	89.9
9	177	45%	0.64	96.3
10	177	26%	0.37	100.0

Sort test customers from highest to lowest predicted risk, cut into 10 equal groups. **Lift** = the group's churn rate ÷ overall churn rate. Decile 1 churned at 98% (lift 1.39). Contacting only the **top 4 deciles (40% of customers) reaches 54% of all churners**; targeting is clearly better than random (which would reach 40%).

10.8 From probability to money: revenue at risk

Probability alone is not enough: a likely-to-churn customer worth ₹500 matters less than a 60%-likely one worth ₹80,000. We compute **revenue at risk = churn probability × historical value** and sort by it. **3,617 customers fall in the 'High' (> 75%) risk band, holding ₹5.8 Cr of historical revenue.**

INTERVIEW ANGLE

'How would you evaluate a churn model before launching?' Temporal validation, AUC/PR-AUC, calibration, decile lift, and ultimately an **A/B test**: treat half the high-risk list, hold out half, compare retention and profit.

'Your model is 90% accurate. Good?' Ask the base rate. If 90% don't churn, a model that never predicts churn is 90% accurate. Use recall/precision/AUC and cost.

'Logistic regression beat boosting; explain.' Signal is simple (recency), small data, boosting adds variance; simpler is also explainable to marketing.

'How would you pick the probability threshold?' Maximise expected profit: benefit of saving a churner × recall vs cost of contacting × false positives.

COMMON MISTAKE

Honest limitations: the label definition (180 days) is a business choice; we evaluated on a random hold-out inside one time cut (production should also test on a later period); probabilities were not calibrated; synthetic data is cleaner than real behaviour.

11. Marketing efficiency: CAC, LTV and payback

Marketing asks 'which channel is cheapest?'. A better question is **'which channel returns the most profit per rupee spent?'** because a cheap channel whose customers leave fast can be a bad deal.

11.1 The three numbers

```
CAC = channel marketing spend ÷ customers acquired through that channel
```

```
LTV (12-month) = average gross profit per customer in the 365 days after sign-up
```

```
LTV:CAC = LTV ÷ CAC Payback (months) = CAC ÷ (LTV ÷ 12)
```

WORKED EXAMPLE

Referral: spend ₹8.68 lakh for 1,116 customers → **CAC = ₹778**. Average 12-month gross profit per customer ₹9,725 → **LTV:CAC = 12.5×**; payback = 778 ÷ (9,725 ÷ 12) = **0.96 months**.

Paid Search: spend ₹59.9 lakh for 2,013 customers → CAC ₹2,977; LTV ₹7,489 → **2.5×**; payback **4.8 months**.

11.2 Why only 'mature' customers for LTV

A customer who signed up last month has not had 12 months to buy, so including them would understate LTV. We compute LTV only for customers with a **full 12-month window** (sign-up on or before 31-Dec-2024). This is the same right-censoring idea as in cohorts.

11.3 Results

Channel	Customers	CAC ₹	12-m LTV ₹	% repeat	LTV:CAC	Payback (mo)
Referral	1,116	778	9,724	72	12.5	1.0
Organic Search	2,469	796	8,927	70	11.2	1.1
Email Marketing	751	1,419	7,697	65	5.4	2.2
Affiliate	881	1,832	6,741	59	3.7	3.3
Social Media	1,770	2,529	7,979	61	3.2	3.8
Paid Search	2,013	2,977	7,489	60	2.5	4.8

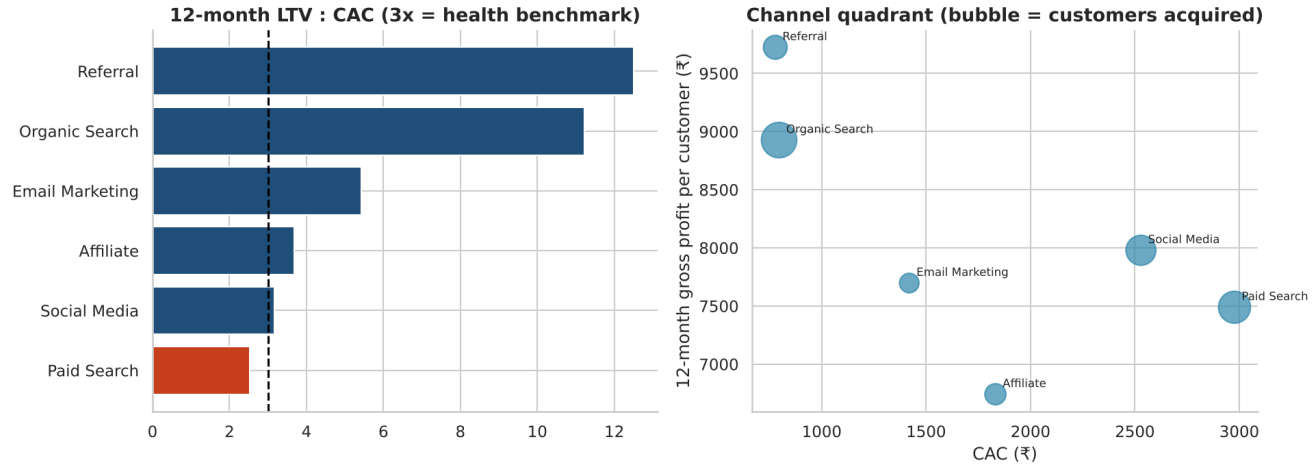


Figure 11.1 - Left: LTV:CAC by channel (red = below the 3× benchmark). Right: CAC vs LTV quadrant.

Referral and Organic Search deliver ≈11-12× with payback in about a month. **Paid Search** is the only channel below the 3× benchmark (2.5×) and Social Media is borderline (3.2×). Customers from Referral also repeat more (72% vs 60%). Recommendation: scale Referral, cap Paid Search CAC, reallocate part of its budget.

COMMON MISTAKE

Caveats that show seniority: (1) CAC here is channel-level and mixes fixed and variable spend; (2) real attribution is messy: customers see several channels (last-click vs multi-touch); (3) LTV here is a 12-month profit proxy, not a probabilistic lifetime model (BG/NBD, Pareto/NBD); (4) the synthetic data bakes in channel differences.

INTERVIEW ANGLE

'What is a good LTV:CAC?' Rule of thumb ≥ 3× with payback < 12 months, but it depends on cash constraints and margin structure.

'CAC went down over time, why?' Fixed spend amortises over more customers (blended CAC fell from ₹2,441 in 2023 to ₹1,536 in 2025); I would also check mix shift towards cheaper channels.

'How would you test if Paid Search is really worse?' Geo or time-based holdout experiment, or incrementality test, since attributed channels overlap.

12. Market basket analysis

Which products are bought together? The answer powers 'frequently bought together' widgets, bundle offers and promotions. The technique is called **association rule mining**. We implement it from scratch (counting pairs); no special library is needed.

12.1 Three metrics

Metric	Question it answers	Formula
Support(A,B)	How common is the pair?	orders with both ÷ all orders
Confidence(A→B)	If A is in the basket, how often is B too?	orders with both ÷ orders with A
Lift(A→B)	Is B bought with A *more than chance*?	confidence ÷ P(B) (1 = no relation, > 1 = real affinity)

WORKED EXAMPLE

10 orders. A (laptop) is in 4, B (accessory) in 5, both in 3.

Support = 3/10 = 0.30. Confidence(A→B) = 3/4 = **0.75**. P(B) = 5/10 = 0.50. Lift = 0.75 ÷ 0.50 = **1.5**: buyers of A are 1.5× more likely to buy B than a random customer.

COMMON MISTAKE

High confidence alone is **not** interesting: if 90% of all orders contain milk, 'bread → milk' has 90% confidence by chance. **Lift** corrects for popularity.

12.2 Results (sub-category level)

If bought	Also bought	Orders together	confidence	lift
Laptops	Laptop Accessories	430	47%	13.2
Laptop Accessories	Laptops	430	43%	13.2
Headphones	Smartphones	813	66%	7.7
Smartphones	Headphones	813	34%	7.7
Fiction	Art Supplies	328	13%	5.3
Art Supplies	Fiction	328	46%	5.3
Fashion Accessories	Women's Wear	821	41%	4.6
Women's Wear	Fashion Accessories	821	32%	4.6

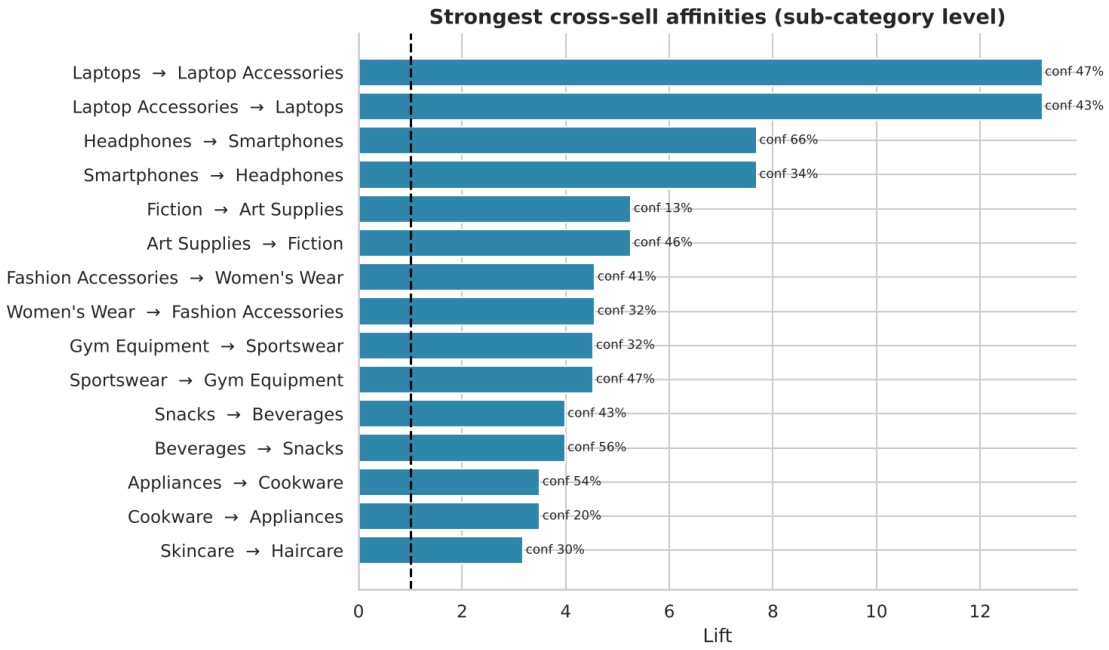


Figure 12.1 - Strongest cross-sell pairs by lift, with confidence labels.

Laptops → Laptop Accessories: lift 13.2×, confidence 47%. Nearly half of laptop orders include an accessory. **Headphones → Smartphones: 66% confidence, lift 7.7×.** Action: bundle offers and cart suggestions on these pairs; accessories carry healthier margins than the laptops themselves (Electronics margin is only 22%).

```
basket = f.groupby("order_id")["sub_category"].agg(lambda s: sorted(set(s)))      # one
    list per order
item = Counter(i for b in basket for i in b)                                        #
    singles
pair = Counter(p for b in basket for p in itertools.combinations(b, 2))            #
    pairs
conf = pair[(a, c)] / item[a];   lift = conf / (item[c] / N)
```

Core of 07_market_basket.py: counting is exact and fast at this scale.

INTERVIEW ANGLE

'How would you build 'customers also bought' for 10 million orders?' Same metrics but computed in Spark/SQL with minimum-support thresholds, FP-Growth/Apriori, or item-embedding recommenders; filter low-support noise.

'Does high lift mean causation?' No; it is co-occurrence. Test promotions with an experiment.

13. Statistics and hypothesis testing

A chart can **look** different without being **really** different: random noise alone creates differences. **Hypothesis testing** gives a disciplined way to ask 'could this difference just be luck?'

13.1 The logic, step by step

1. **Null hypothesis (H0):** the boring claim, 'there is no difference / no effect'. e.g. COD and prepaid orders cancel at the same rate.
2. **Alternative (H1):** there is a difference.
3. Compute a **test statistic** that measures how far the data is from what H0 predicts.
4. Compute the **p-value**: the probability of seeing a difference at least this large **if H0 were true**.
5. If $p < 0.05$ (the conventional **significance level α**) we **reject H0**: the difference is unlikely to be luck.

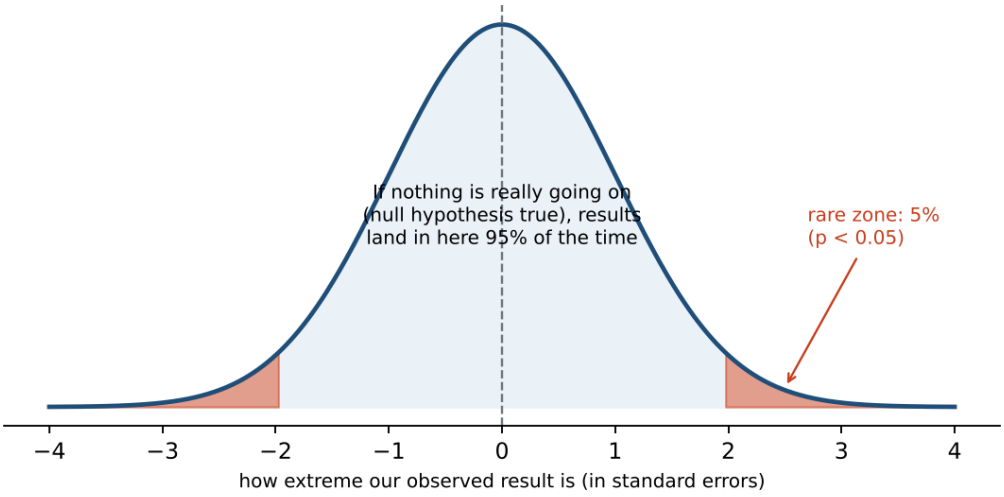


Figure 13.1 - If the null is true, extreme results are rare; a tiny p-value says our result sits in that rare zone.

ANALOGY

A courtroom: the null hypothesis is 'innocent until proven guilty'. The p-value says how surprising the evidence would be if the defendant were innocent. Very surprising → we reject innocence. It does **not** say how big the crime is, nor the probability that H0 is true.

COMMON MISTAKE

p-value misreadings: it is NOT 'the probability the null is true' and NOT the size of the effect.

Significance ≠ importance. With 29,000 orders even a trivial gap can be significant. Always report the **effect size** (how big) and a **confidence interval** (how uncertain).

13.2 Errors

	H0 actually true	H0 actually false
We reject H0	**Type I error** (false alarm), probability = α	Correct (power)
We keep H0	Correct	**Type II error** (missed effect)

Running many tests inflates false alarms (test 20 things at α = 5% and one will be 'significant' by luck). Pre-state your hypotheses or adjust (Bonferroni).

13.3 Choosing the test

Situation	Test	Used here for
Two categories vs yes/no outcome (rates)	**Chi-square** test of independence / 2-proportion test	COD vs prepaid cancellation; discount band vs returns
Number across 3+ groups, skewed data	**Kruskal-Wallis** (non-parametric ANOVA)	Delivery days by region; LTV by channel
Number across 2 groups, skewed data	**Mann-Whitney U**	Festive vs non-festive order value
Two numeric variables, monotonic relationship	**Spearman** correlation	Discount % vs units per line
Any statistic, small/odd data	**Bootstrap** confidence interval	Difference in AOV
Roughly normal numbers, 2 groups	t-test	(not used: money is skewed)

BEGINNER TIP

Non-parametric tests (Mann-Whitney, Kruskal, Spearman) use ranks rather than raw values, so skewed money data and outliers do not break them. Prefer them for revenue, days and counts.

13.4 A hand-calculable chi-square example

WORKED EXAMPLE

Illustrative numbers (not the real data): 1,000 COD orders with 90 cancelled; 4,000 prepaid orders with 130 cancelled. Overall cancel rate = 220 ÷ 5,000 = 4.4%.

If H0 were true, COD would expect 44 cancellations (4.4% of 1,000) and prepaid 176. Chi-square = Σ (observed − expected)² ÷ expected over the four cells:

$(90-44)^2/44 + (910-956)^2/956 + (130-176)^2/176 + (3870-3824)^2/3824 = 48.1 + 2.2 + 12.0 + 0.6 =$ **62.9**. Critical value for 5% with 1 degree of freedom is 3.84, so 62.9 is wildly beyond it: $p \approx 10^{-15}$ → reject H0.

13.5 Confidence intervals and the bootstrap

A **95% confidence interval** is a range built so that, over many repeats of the study, 95% of such ranges would contain the true value. Narrow interval = precise estimate. The **bootstrap** builds one by resampling: draw rows with replacement from the data thousands of times, recompute the statistic each time, and take the middle 95% of results.

```
boot = [rng.choice(a, len(a)).mean() - rng.choice(b, len(b)).mean() for _ in
    range(2000)]
lo, hi = np.percentile(boot, [2.5, 97.5])     # 95% interval for the difference in
    average order value
```

13.6 Our six tests

test	p-value	Effect size / detail	conclusion
COD vs prepaid cancellation	6.2e-73	COD 8.9% vs 3.3%; diff 95% CI [4.8, 6.4] pp	REJECT H0 (significant)
Discount band vs return rate	9e-35	return % by band: 0%=8.3, 1-10%=8.8, 11-20%=8.6, >20%=12.9	REJECT H0 (significant)
Delivery days across regions	0	median days: Central=6, East=6, North=5, North-East=8, South=5, West=5	REJECT H0 (significant)

test	p-value	Effect size / detail	conclusion
12-m gross profit per customer by channel	3.5e-07	median ₹: Referral=5,393, Organic=5,250, Email=4,710, Affiliate=4,344, Social=4,226, Paid=4,027	REJECT H0 (significant)
Festive vs non-festive AOV	1.1e-08	ΔAOV ₹-529, bootstrap 95% CI [₹-983, ₹-89]	REJECT H0 (significant)
Discount % vs units per line (Spearman)	0.28	rho=0.004	Fail to reject H0

13.7 What the results mean for the business

- **COD cancellation:** COD 8.9% vs 3.3% prepaid, gap 95% CI [4.8, 6.4] pp, p ≈ 10^{-73}. Strong, precise, practically big → nudge prepaid or confirm COD orders.
- **Discount vs returns:** the > 20% discount band returns 12.9% of lines vs ≈8.5% elsewhere (p ≈ 10^{-35}). Deep discounts attract returners.
- **Discount vs units:** Spearman ρ = 0.004, p = 0.29: **no evidence** that deeper discounts sell more units per line. Failing to reject is a finding, not a failure.
- **Festive AOV:** average order value is ₹529 *lower* in Oct-Nov (bootstrap CI −983 to −89), consistent with heavier festive discounting (not proven causal).

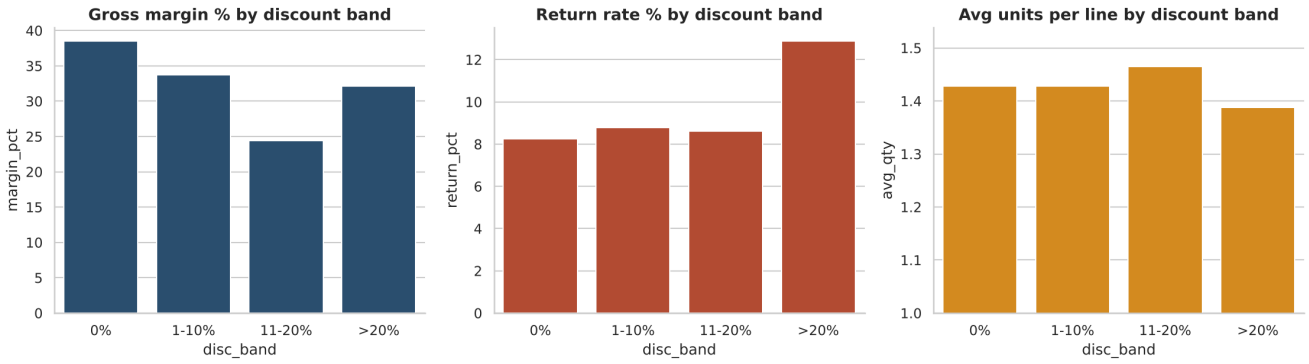


Figure 13.2 - Margin %, return rate % and units per line by discount band.

Discount band	lines	Avg units/line	Return %	Margin %
0%	16,398	1.43	8.3	38.5
1-10%	20,152	1.43	8.8	33.8
11-20%	15,370	1.47	8.6	24.4
>20%	8,199	1.39	12.9	32.1

COMMON MISTAKE

Confounding (Simpson-type trap): festive months have both more discounting and more volume, so a naive comparison would credit discounts for the festive volume. Control for it (compare within category/month) or use an experiment. The ρ ≈ 0 above is across all lines; a real experiment would confirm it. The >20% band's margin looks higher (32%) than the 11-20% band only because of **product mix** (caps keep Electronics under 12%, so deep-discount lines are mostly high-margin Fashion/Beauty).

13.8 The discount-cap what-if

If deeper discounts do not buy volume, capping them gives money back. For each cap we re-price every line as `gross_sales × (1 − min(discount, cap))` and sum the extra profit, assuming **zero volume loss**.

Cap %	Profit uplift ₹	% of gross profit
0	26,754,750	34.65
5	16,347,846	21.17
10	8,387,185	10.86
15	4,662,539	6.04
20	2,881,621	3.73
30	1,030,631	1.33

A 10% cap ≈ **+₹83.9 lakh (+10.9% of gross profit)**. This is an upper bound: the zero-volume-loss assumption must be validated by an **A/B test** before any roll-out.

> **INTERVIEW ANGLE**
>
> **'Explain a p-value to a non-technical manager.'** 'If the two payment methods really cancelled equally often, we would almost never see a gap this large by chance. So the gap is real.'
>
> **'p = 0.04: is the effect important?'** Not necessarily; check effect size, interval and cost/benefit.
>
> **'Why not a t-test?'** Revenue and delivery days are skewed with outliers; rank-based tests are safer, and the bootstrap needs no distribution assumption.
>
> **'How would you test the discount cap?'** Randomise customers/regions/time into cap vs control; primary metric profit per visitor; guardrails on conversion and returns; compute sample size beforehand.

14. Revenue forecasting

Finance and supply chain need a defensible outlook: how much inventory, how much cash? A **forecast** predicts future values of a time series from its past.

14.1 Anatomy of a time series

Component	Meaning	KartNova
Trend	Long-run direction	Upward, slowing growth
Seasonality	Repeating calendar pattern	Oct-Nov peaks, Feb dip
Noise	Random wiggles no model can predict	Month-to-month randomness

14.2 Our three candidate models

Model	Idea
A. Seasonal naive × growth	Next March = last March × recent growth. Simple benchmark
B. Log-trend + month effects (selected)	Regress log(revenue) on log(time) and 12 month dummies: the log lets growth be multiplicative and slow down; month dummies capture seasonality
C. Linear trend + month effects	Same but straight-line trend: overshoots when growth slows

Taking **logs** turns multiplicative effects (+20% in a festive month) into additive ones, which linear regression handles naturally, and guarantees forecasts stay positive.

14.3 Choosing by backtest, not by fit

A model that fits the past perfectly may forecast badly. **Rolling-origin backtesting** mimics real use: train on data up to a date, forecast the next 6 months, compare with what actually happened, then slide the origin forward and repeat. Choose the model with the lowest average error.

```
MAPE = average of |forecast − actual| ÷ actual × 100%
```

Train ends	C: linear	B: log-trend	A: seasonal naive
2024-06	28.2	19.97	325.74
2024-09	34.95	13.85	135.64
2024-12	20.42	15.03	47.33
2025-03	23.01	13.43	27.88
2025-06	46.12	10.08	34.49
MEAN	30.54	14.47	114.22

The log-trend model wins (**MAPE 14.5%**) vs 30.5% for linear and 114% for seasonal naive (the naive benchmark fails because the business was growing explosively from a tiny base). We train on data from April 2023 to drop the very early ramp-up months.

14.4 The forecast

month	Forecast ₹	80% low	80% high
2026-01-01	12,319,777	10,337,620	14,681,996
2026-02-01	10,042,074	8,426,382	11,967,562
2026-03-01	10,927,752	9,169,562	13,023,063
2026-04-01	13,357,203	11,208,132	15,918,340
2026-05-01	12,214,924	10,249,637	14,557,039
2026-06-01	11,990,885	10,061,644	14,290,042

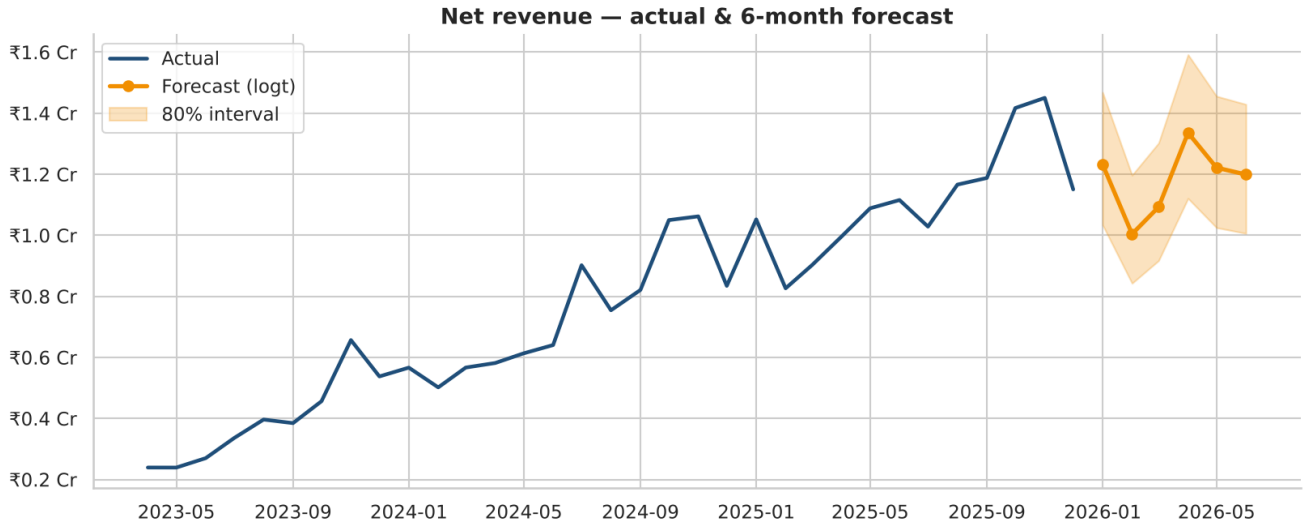


Figure 14.1 - Actual monthly revenue and the 6-month forecast with an 80% prediction interval.

An **80% prediction interval** says: we expect the actual value inside the band 8 times out of 10. It is built from the residual spread of the model. Wide bands honestly communicate uncertainty (roughly −16% / +19%).

COMMON MISTAKE

Caveats: only ~33 monthly points for a model with 13 parameters; no promotions, price, or marketing as inputs; the trend assumption (decelerating log growth) may break. In production: re-fit monthly, track forecast error, and consider ETS/ARIMA/Prophet or gradient-boosted models with calendar and marketing features.

Forecasts are tools for planning ranges, not promises.

INTERVIEW ANGLE

'How do you evaluate a forecast?' Out-of-sample error (MAPE/MAE/RMSE) over rolling origins vs a naive benchmark; prediction-interval coverage.

'MAPE problems?' Explodes when actuals are near zero and is asymmetric; consider sMAPE, MASE or WAPE.

'Your forecast missed by 25% in a month: what do you do?' Diagnose: was there an event (promotion, stock-out)? Is it bias or noise? Add the driver or widen intervals; never silently re-tune on the miss.

15. SQL from zero, and every query in the project

SQL (Structured Query Language) is the language for asking questions of databases. It is the single most-tested analyst skill. You describe **what** you want (not how to compute it) and the database figures out how.

ANALOGY

Asking a librarian: 'Give me the titles of all books published after 2015, grouped by author, with the author who has the most at the top.' You do not tell the librarian which shelves to walk; SQL is that polite request in a precise format.

15.1 The anatomy of a query

```
SELECT   category, SUM(net_revenue) AS revenue      -- 5. which columns / calculations
    to show
FROM     vw_order_lines                              -- 1. which table
WHERE    order_year = '2025'                         -- 2. filter rows BEFORE grouping
GROUP BY category                                    -- 3. make one row per category
HAVING   SUM(net_revenue) > 1000000                  -- 4. filter groups AFTER grouping
ORDER BY revenue DESC                                -- 6. sort
LIMIT    5;                                          -- 7. keep top 5
```

The numbers show the LOGICAL order the database evaluates clauses, not the order you type them.

KEY IDEA

Order of evaluation: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That is why you cannot use a SELECT alias inside WHERE (it does not exist yet) and why WHERE filters rows but HAVING filters groups.

15.2 Essential building blocks

Need	SQL	Note
Count rows / distinct	`COUNT(*)`, `COUNT(DISTINCT customer_id)`	COUNT(col) ignores NULLs
Totals	`SUM`, `AVG`, `MIN`, `MAX`	Ignore NULLs
If/else	`CASE WHEN x > 10 THEN 'A' ELSE 'B' END`	Also used for conditional counting
Nulls	`IS NULL`, `COALESCE(x, 0)`	`= NULL` never works; NULL is 'unknown'
Text match	`LIKE 'Lap%'`	% = any characters
Lists	`IN ('UPI','Wallet')`	Shorthand for many ORs
Combine results	`UNION ALL` (keeps duplicates), `UNION` (removes)	Same columns required
Dates (SQLite)	`strftime('%Y-%m', d)`, `julianday(d2) - julianday(d1)`, `date(d, '+365 day')`	Dialects differ, see 15.7

COMMON MISTAKE

NULL logic: `NULL = NULL` is not true. `WHERE x <> 5` drops rows where x is NULL. `AVG` ignores NULLs, so averaging a column that contains NULLs divides by fewer rows than you may expect.

Integer division: in some databases `1/2 = 0`. Multiply by `1.0` first (we use `100.0 * a / b`).

15.3 JOINs: combining tables

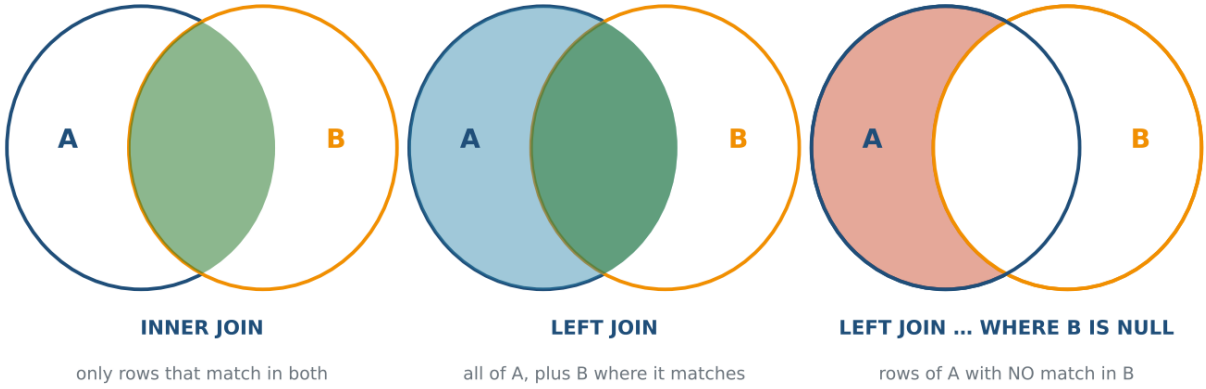


Figure 15.1 - INNER keeps matches; LEFT keeps all of the left table; LEFT + IS NULL finds rows with no match.

```
SELECT o.order_id, c.city
FROM   orders o
JOIN   customers c ON c.customer_id = o.customer_id        -- INNER JOIN: only orders
    with a known customer

-- customers who NEVER ordered (anti-join)
SELECT c.customer_id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE  o.order_id IS NULL;
```

15.4 CTEs: readable multi-step logic

A **CTE (Common Table Expression)** is a named temporary result defined with `WITH`. It lets you build an answer in steps, each step readable, instead of nesting subqueries inside subqueries. Every non-trivial query in this project uses them.

```
WITH monthly AS (                                -- step 1: revenue per month
    SELECT order_month, SUM(net_revenue) AS revenue
    FROM vw_order_lines GROUP BY order_month)
SELECT order_month, revenue,                     -- step 2: use step 1
       revenue / LAG(revenue) OVER (ORDER BY order_month) - 1 AS mom_growth
FROM monthly;
```

15.5 Window functions: the senior-analyst skill

GROUP BY region → rows COLLAPSE
(input has 5 rows: 40,25 | 20,10,30)

region	SUM(revenue)
West	65
South	60

Output: 1 row per group.
City detail is gone.

SUM(revenue) OVER (PARTITION BY region) → rows KEPT

region	city	revenue	region total	share
West	Mumbai	40	65	62%
West	Pune	25	65	38%
South	Chennai	20	60	33%
South	Kochi	10	60	17%
South	Hyderabad	30	60	50%

Every row survives; the group total rides along,
so 'share of region' is one division away.

Figure 15.2 - GROUP BY collapses rows. A window function keeps every row and adds a calculation computed over a related set of rows.

A **window function** performs a calculation across a set of rows related to the current row **without collapsing them**. Syntax: `function() OVER (PARTITION BY ... ORDER BY ... ROWS ...)`. `PARTITION BY` = which rows form the window (like GROUP BY but non-collapsing); `ORDER BY` = ordering inside it.

Family	Functions	Typical use	In this project
Ranking	`ROW_NUMBER`, `RANK`, `DENSE_RANK`	Top-N per group, dedupe	q06 top 3 products per category
Offset	`LAG`, `LEAD`, `FIRST_VALUE`	Compare to previous row / first row	q02 MoM & YoY, q10 cohort size
Buckets	`NTILE(n)`	Quintiles/deciles	q09 RFM scores, q12 deciles
Running / share	`SUM() OVER (ORDER BY ...)`, `SUM() OVER ()`	Cumulative totals, % of total	q03 YTD, q05 Pareto, q04 share

WORKED EXAMPLE

Sales by month: Jan 100, Feb 120, Mar 90. `LAG(revenue)` gives the previous row: Jan NULL, Feb 100, Mar 120. Growth = revenue/LAG − 1 → Feb +20%, Mar −25%.

`RANK` vs `ROW_NUMBER` vs `DENSE_RANK` on values 90, 90, 80: ROW_NUMBER 1,2,3; RANK 1,1,3 (skips); DENSE_RANK 1,1,2 (no gap).

COMMON MISTAKE

Default window frame when you add `ORDER BY` is **RANGE UNBOUNDED PRECEDING to CURRENT ROW** (running total, and ties are included together). If you want exactly the last 3 rows write `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` (we do this for the 3-month moving average).

15.6 The project's database design

```
CREATE TABLE orders (
    order_id TEXT PRIMARY KEY,
    customer_id TEXT NOT NULL REFERENCES customers(customer_id),      -- foreign key
    order_status TEXT CHECK (order_status IN ('Delivered','Cancelled')),
    ...);
CREATE INDEX ix_orders_customer ON orders(customer_id);              -- speeds up
    joins/filters on this column

CREATE VIEW vw_order_lines AS ...   -- the analytical base: money columns defined ONCE
```

- **PRIMARY KEY / FOREIGN KEY / CHECK / NOT NULL** are **constraints**: the database itself refuses bad data (e.g. an order for a non-existent customer).
- **Index**: like a book's index; the database jumps to matches instead of scanning every row. Index columns used in JOIN/WHERE; indexes slow down writes slightly.
- **View**: a saved query that behaves like a table. `vw_order_lines` defines net revenue, refunds and gross profit once, so every query reuses the same business rules.

15.7 Dialect differences (SQLite vs the others)

Task	SQLite (project)	PostgreSQL	SQL Server	MySQL
Month bucket	`strftime('%Y-%m', d)`	`date_trunc('month', d)`	`DATEFROMPARTS(YEAR(d),MONTH(d),1)`	`DATE_FORMAT(d,'%Y-%m')`
Days between	`julianday(a)-julianday(b)`	`a - b`	`DATEDIFF(day, b, a)`	`DATEDIFF(a, b)`
Add days	`date(d,'+365 day')`	`d + INTERVAL '365 day'`	`DATEADD(day,365,d)`	`DATE_ADD(d, INTERVAL 365 DAY)`
Limit rows	`LIMIT n`	`LIMIT n`	`TOP n` / `FETCH`	`LIMIT n`

The logic is identical across dialects; only date functions and a few keywords change. Say this in interviews: 'I write ANSI SQL and translate dates.'

15.8 The project queries, one by one

q01 KPI scorecard by year (aggregation + conditional logic)

```
SELECT order_year, ROUND(SUM(net_revenue),0) AS net_revenue,
       ROUND(100.0 * SUM(gross_profit) / SUM(net_revenue - refund_amt), 2) AS
           gross_margin_pct,
       COUNT(DISTINCT order_id) AS orders,
       ROUND(100.0 * AVG(is_returned), 2) AS return_rate_pct
FROM vw_order_lines GROUP BY order_year ORDER BY order_year;
```

order_year	net_revenue	gross_margin_pct	orders	active_customers	aov	return_rate_pct
2023	38,809,309	31.69	4,022	1,956	9,649	9.5
2024	88,935,118	32.42	9,805	4,204	9,070	8.85
2025	133,807,961	32.38	14,044	5,573	9,528	9.27

Why it matters: this is the numbers that must match Python and Power BI. `AVG(is_returned)` works because the column is 0/1.

q02 Month-over-month and year-over-year growth (LAG)

```
WITH m AS (SELECT order_month, SUM(net_revenue) AS revenue FROM vw_order_lines GROUP BY
    order_month)
SELECT order_month, revenue,
  ROUND(100.0*(revenue / LAG(revenue, 1)  OVER (ORDER BY order_month) - 1), 1) AS
      mom_growth_pct,
  ROUND(100.0*(revenue / LAG(revenue, 12) OVER (ORDER BY order_month) - 1), 1) AS
      yoy_growth_pct,
  ROUND(AVG(revenue) OVER (ORDER BY order_month ROWS BETWEEN 2 PRECEDING AND CURRENT
      ROW), 0) AS revenue_3m_ma
FROM m ORDER BY order_month;
```

`LAG(revenue, 12)` looks 12 rows back = same month last year (YoY removes seasonality); `LAG(…,1)` is the previous month (MoM = momentum).

q05 Pareto and ABC classification (cumulative window)

```
WITH p AS (SELECT product_id, SUM(net_revenue) AS revenue FROM vw_order_lines GROUP BY
    product_id),
     c AS (SELECT *, SUM(revenue) OVER (ORDER BY revenue DESC) / SUM(revenue) OVER () AS
         cum_share FROM p)
SELECT product_id, revenue, ROUND(100*cum_share,1) AS cum_pct,
       CASE WHEN cum_share <= .70 THEN 'A' WHEN cum_share <= .90 THEN 'B' ELSE 'C' END
           AS abc_class
FROM c ORDER BY revenue DESC;
```

`SUM() OVER (ORDER BY revenue DESC)` is a running total from best to worst; `SUM() OVER ()` (empty window) is the grand total; dividing gives the cumulative share.

q06 Top-N per group (ROW_NUMBER + PARTITION BY)

```
WITH p AS (SELECT category, product_name, SUM(net_revenue) AS revenue FROM
    vw_order_lines GROUP BY 1, 2),
     r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS
         rn FROM p)
SELECT * FROM r WHERE rn <= 3;
```

> **INTERVIEW ANGLE**
>
> 'Top 3 products per category' is perhaps the most common SQL interview question. You **cannot** use `LIMIT 3` per group; you rank within each group with a window function and filter on the rank in an outer query (window functions cannot be used directly in WHERE).

q09 RFM in pure SQL (NTILE)

```
WITH base AS (SELECT customer_id, MAX(order_date) AS last_order, COUNT(DISTINCT
    order_id) AS frequency,
                     SUM(net_revenue - refund_amt) AS monetary FROM vw_order_lines GROUP
                         BY customer_id),
scored AS (SELECT *, 6 - NTILE(5) OVER (ORDER BY julianday('2025-12-31') -
    julianday(last_order)) AS r_score,
                     NTILE(5) OVER (ORDER BY frequency, customer_id) AS f_score,
                     NTILE(5) OVER (ORDER BY monetary,  customer_id) AS m_score FROM
                         base)
SELECT CASE WHEN r_score>=4 AND f_score>=4 AND m_score>=4 THEN 'Champions' ... END AS
    segment, COUNT(*) AS customers ...
```

`NTILE(5)` deals rows into 5 equal buckets. `6 − NTILE` reverses the scale for recency. Adding `customer_id` to the ORDER BY makes ties break the same way every run (**deterministic**).

segment	customers	revenue_pct	avg_recency_days	avg_orders
Champions	1,473	42.9	52.0	7.3
Can't Lose Them	645	15.7	526.0	5.4
Loyal Customers	1,116	14.1	161.0	4.7
Needs Attention	1,347	8.9	437.0	1.1
Potential Loyalists	1,069	7.6	126.0	2.0
At Risk	1,009	6.3	535.0	2.5
New / Promising	898	3.4	80.0	1.0
Lost	1,297	1.3	655.0	1.1

q10 Cohort retention (self-referencing CTEs)

```
WITH o AS (SELECT DISTINCT customer_id, order_id, order_month FROM vw_order_lines),
first_m AS (SELECT customer_id, MIN(order_month) AS cohort_month FROM o GROUP BY
    customer_id),
act AS (SELECT f.cohort_month,
         (CAST(substr(o.order_month,1,4) AS INT) - CAST(substr(f.cohort_month,1,4) AS
             INT)) * 12
       + (CAST(substr(o.order_month,6,2) AS INT) - CAST(substr(f.cohort_month,6,2) AS
           INT)) AS month_n,
         COUNT(DISTINCT o.customer_id) AS active_customers
        FROM o JOIN first_m f USING (customer_id) GROUP BY 1, 2)
SELECT cohort_month, month_n, active_customers,
       100.0 * active_customers / FIRST_VALUE(active_customers) OVER (PARTITION BY
           cohort_month ORDER BY month_n) AS retention_pct
FROM act WHERE month_n <= 12;
```

Month index = (year difference × 12) + month difference, done with string slicing so it works in any dialect. `FIRST_VALUE` picks month-0 customers as the denominator within each cohort.

q11 Repeat-purchase funnel and gap between orders

```
SELECT order_seq AS nth_order, COUNT(*) AS customers_reaching, ROUND(AVG(gap_days),1) AS
     avg_days_since_previous,
       ROUND(100.0*COUNT(*)/FIRST_VALUE(COUNT(*)) OVER (ORDER BY order_seq),1) AS
           pct_of_first_time_buyers
FROM (SELECT customer_id,
        julianday(order_date) - julianday(LAG(order_date) OVER (PARTITION BY customer_id
             ORDER BY order_date, order_id)) AS gap_days,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS
            order_seq
      FROM (SELECT DISTINCT customer_id, order_id, order_date FROM vw_order_lines))
          WHERE order_seq <= 8 GROUP BY order_seq;
```

nth_order	customers_reaching	avg_days_since_previous	pct_of_first_time_buyers
1	8,854	nan	100.0
2	5,527	94.6	62.4
3	3,573	79.9	40.4
4	2,501	69.1	28.2
5	1,765	66.0	19.9

62.4% of customers place a 2nd order (after 95 days on average); 40.4% reach a 3rd. Each step is a drop-off in the loyalty funnel.

q12 Customer revenue concentration (deciles)

decile	customers	pct_of_revenue	cumulative_pct
1	886	48.8	48.8
2	886	20.9	69.7
3	886	11.9	81.6
4	886	6.9	88.5
5	885	4.4	92.9

The top 10% of customers deliver 48.8% of revenue; the top 30% deliver 81.6%. Concentration risk: losing a few VIPs hurts disproportionately.

q14 CAC and LTV by channel (joining facts at different grains)

```
WITH sp AS (SELECT channel, SUM(marketing_spend) AS spend FROM marketing_spend GROUP BY
    channel),
     nc AS (SELECT acquisition_channel AS channel, COUNT(*) AS new_customers FROM
         customers GROUP BY 1),
     mat AS (SELECT customer_id, acquisition_channel AS channel, signup_date FROM
         customers WHERE signup_date <= '2024-12-31'),
     ltv AS (SELECT m.channel, SUM(v.gross_profit)*1.0/COUNT(DISTINCT m.customer_id) AS
         ltv12_gp
             FROM mat m LEFT JOIN vw_order_lines v ON v.customer_id = m.customer_id
                                                  AND v.order_date <= date(m.signup_date,
                                                       '+365 day')
             GROUP BY m.channel)
SELECT sp.channel, sp.spend/nc.new_customers AS cac, ltv.ltv12_gp,
    ltv.ltv12_gp/(sp.spend/nc.new_customers) AS ltv_to_cac
FROM sp JOIN nc USING (channel) JOIN ltv USING (channel);
```

COMMON MISTAKE

Aggregate before you join. Spend is per channel-month, customers per customer; joining them raw would multiply rows. Each CTE reduces its table to one row per channel first, then they are joined safely: this is the standard fix for 'fan-out' inflation.

q18 Market basket in SQL (self-join)

```
WITH basket AS (SELECT DISTINCT order_id, sub_category FROM vw_order_lines),
     pair AS (SELECT a.sub_category AS antecedent, b.sub_category AS consequent,
         COUNT(*) AS together
              FROM basket a JOIN basket b ON a.order_id = b.order_id AND a.sub_category
                  <> b.sub_category
              GROUP BY 1, 2 HAVING COUNT(*) >= 40) ...
```

Joining the basket table **to itself** on the same order produces every product pair; counting them gives pair frequency. `a.sub_category <> b.sub_category` removes pairing a product with itself.

Data-quality assertions (05_data_quality_checks.sql)

Every query returns **failures**, so 0 means healthy: orphan foreign keys, delivery before order, discount outside 0-100, price below cost, a cancelled order that has a return. Putting these in SQL means the checks can run automatically after every data load.

15.9 Classic SQL interview patterns (with the answer idea)

Question	Pattern
Second-highest value / Nth highest	`DENSE_RANK() OVER (ORDER BY x DESC)` then `WHERE rank = N`
Top N per group	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)` then filter
Find duplicates	`GROUP BY key HAVING COUNT(*) > 1`
Delete duplicates keep one	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY id)` and delete where rn > 1
Running total / moving average	`SUM(x) OVER (ORDER BY d ROWS BETWEEN n PRECEDING AND CURRENT ROW)`
Month-over-month growth	`LAG()` in a CTE of monthly totals
Customers with no orders	`LEFT JOIN ... WHERE right.key IS NULL` or `NOT EXISTS`
Retention / returning users	Self-join or cohort CTE on first-order month
Consecutive days / streaks	Subtract `ROW_NUMBER()` from the date; equal results = same streak (gaps-and-islands)
Pivot rows to columns	`SUM(CASE WHEN cat='A' THEN x END) AS a, ...`
Percent of total	`x / SUM(x) OVER ()`
Median	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)` (or ROW_NUMBER trick)

15.10 Performance basics

- Filter early (WHERE before joins/aggregation) and select only needed columns (avoid `SELECT *`).
- Index join/filter columns; check the plan with `EXPLAIN`.
- Avoid functions on indexed columns in WHERE (`WHERE strftime('%Y', d) = '2025'` blocks the index; prefer `d >= '2025-01-01' AND d < '2026-01-01'`).
- `COUNT(DISTINCT)` and sorting are expensive on huge tables; pre-aggregate when possible.
- `UNION ALL` is faster than `UNION` (no dedupe) when duplicates are impossible.

INTERVIEW ANGLE

'WHERE vs HAVING?' WHERE filters rows before grouping; HAVING filters aggregated groups.

'RANK vs DENSE_RANK vs ROW_NUMBER?' See Section 15.5 example.

'INNER vs LEFT JOIN: which changes row counts?' Inner can drop rows; left keeps all left rows; both can multiply rows if the right side has duplicates.

'How do you debug a query whose total does not match the dashboard?' Check grain/duplicates after joins, filters (status, dates), NULL handling, and compare totals step by step (CTE by CTE).

16. Power BI and DAX from zero

16.1 What is Power BI?

Power BI is Microsoft's business-intelligence tool: it connects to data, models it, and presents interactive dashboards that non-technical people can click and filter. It has three parts you should name in an interview:

Part	What it is	Who uses it
Power BI Desktop (free)	Windows app where you load data, build the model, write DAX and design reports	Analyst / BI developer
Power BI Service (cloud)	Where reports are published, shared, scheduled for refresh, with security	Everyone
Gateway	A bridge letting the cloud refresh data held on-premises	IT / BI developer

16.2 The Power BI workflow

1. **Get data** (CSV, SQL, Excel, web...).
2. **Power Query**: clean and shape the data (the 'ETL' step: Extract, Transform, Load). Language: **M**.
3. **Model**: relate tables into a star schema.
4. **DAX**: write measures and calculated columns (the calculation language).
5. **Visuals**: charts, tables, slicers on report pages.
6. **Publish & share**: Service, scheduled refresh, row-level security.

> **ANALOGY**
>
> Think of a restaurant. Power Query is the kitchen prep (washing, chopping); the data model is the organised pantry; DAX is the recipe book; the report is the plated dish the customer sees.

16.3 Power Query (M): prepare the data

Power Query records each transformation step (change type, filter rows, merge tables, add column) in order, so refreshing replays the exact same steps on new data. Our M queries (`power_query_m.pq`):

Query	Purpose
`DataFolder` (parameter)	One text value holding the folder path. Change it once and every query repoints; no hard-coded paths
`fnLoadCsv` (function)	Reusable CSV loader (DRY: Don't Repeat Yourself)
`Fact_Orders`	One row per order, all statuses (needed for cancellation and delivery)
`Fact_Sales`	One row per **delivered** order line, with product cost and return info merged in, plus money columns (net revenue, refund, COGS, gross profit)
`Dim_Customer`, `Dim_Product`	Descriptive tables
`Fact_Marketing`, `Dim_Channel`	Monthly spend; a small conformed channel table so spend and customers share one slicer

```
// Fact_Sales (excerpt): money columns computed once at refresh
GrossSales = Table.AddColumn(Typed0, "gross_sales", each [quantity] * [unit_price], type
     number),
NetRev     = Table.AddColumn(GrossSales, "net_revenue", each [gross_sales] * (1 -
    [discount_pct] / 100), type number),
Refund     = Table.AddColumn(IsRet, "refund_amt", each if [refund_pct] = null then 0
    else [net_revenue] * [refund_pct], type number)
```

KEY IDEA

Rule of thumb: do **row-level maths** (one value per row) in Power Query/SQL; do **aggregation and context-aware logic** (totals that react to slicers, growth, ranking) in DAX measures. Row-level columns are computed once and compress well; measures are computed live in the visual's context.

16.4 The data model

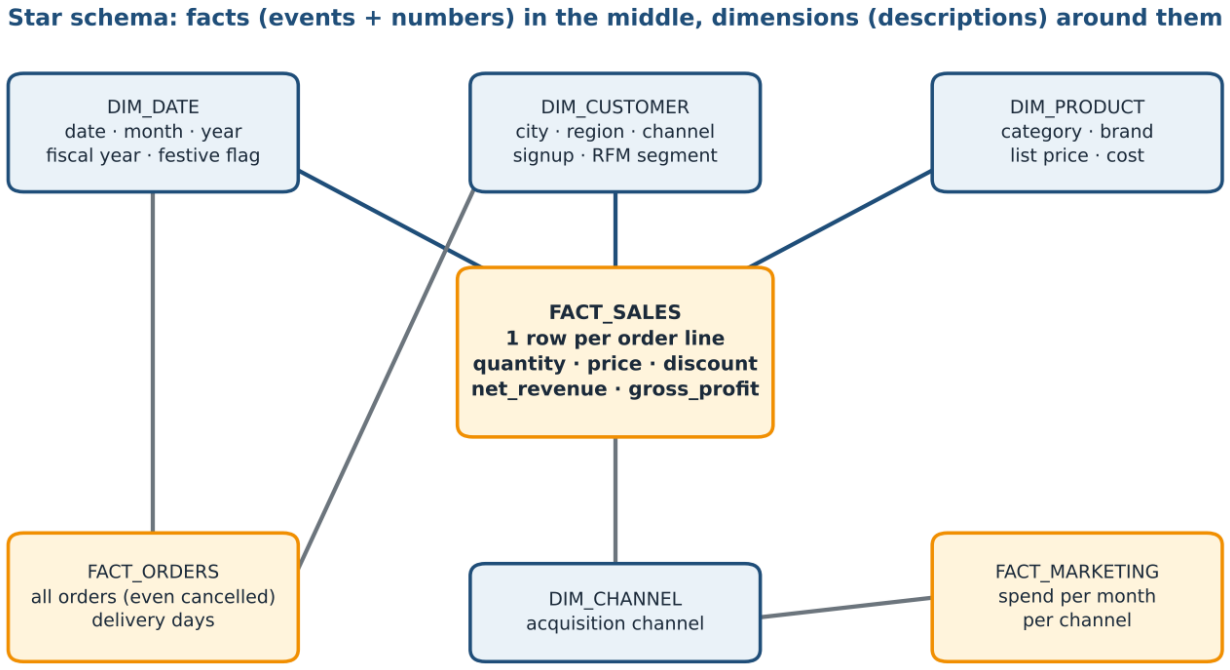


Figure 16.1 - The Power BI model is the same star schema as the database.

Setting	Our choice	Reason
Cardinality	Many-to-one (fact → dimension)	Many order lines per product/customer/date
Filter direction	**Single** (dimension filters fact)	Bi-directional filters cause ambiguity and slow queries
Date table	Continuous calendar, **marked as date table**	Time-intelligence functions require it
Hidden columns	Keys and raw money columns	Users should use measures, not raw columns
Two fact tables	Fact_Sales (delivered lines) and Fact_Orders (all orders)	Cancellation needs cancelled orders; revenue must exclude them

16.5 DAX fundamentals (read this twice)

DAX (Data Analysis Expressions) is the formula language of Power BI. It looks like Excel formulas but works on whole columns and tables, and its behaviour depends on **context**. Master context and DAX stops being mysterious.

Calculated column vs measure

	Calculated column	Measure
Computed when	At refresh, row by row	At query time, when a visual needs it
Stored?	Yes (uses memory)	No (just a formula)
Reacts to slicers?	No (frozen)	**Yes**
Can be used as axis/slicer?	Yes	No
Use for	Attributes: RFM segment, cohort month	KPIs: revenue, rates, growth

Filter context: the key idea

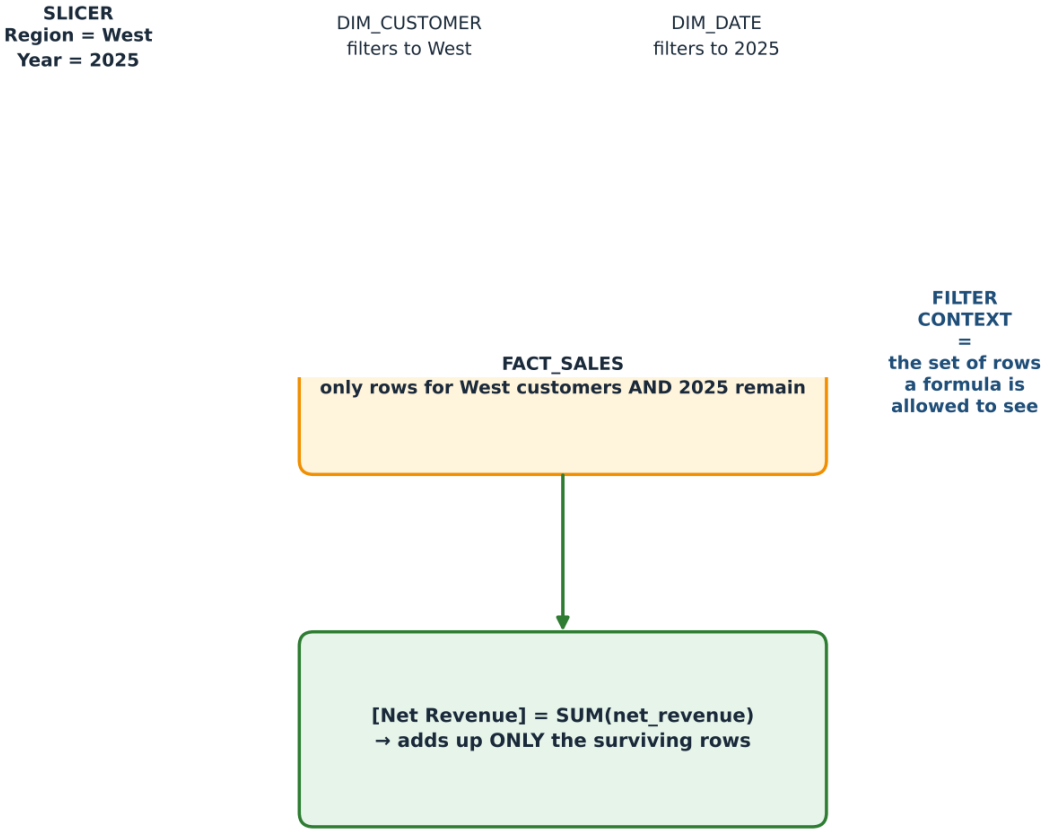


Figure 16.2 - Slicers and visual rows create a filter context; the measure sees only the surviving rows.

When a visual shows 'Net Revenue' for 'West' in '2025', the formula `SUM(net_revenue)` does not change; **the rows it can see change**. A bar for Fashion sees only Fashion rows; a card with no filters sees all rows. That set of visible rows is the **filter context**. One measure, thousands of different answers.

Row context

Row context means 'the current row' inside an iteration or a calculated column. `[quantity] * [unit_price]` in a calculated column works row by row. Aggregators like `SUMX(Fact_Sales, [qty]*[price])` create a row context for each row, then add up.

CALCULATE: the most important function

```
CALCULATE( <expression>, <filter1>, <filter2>, ... )
```

`CALCULATE` evaluates an expression **after modifying the filter context**. It is how you say 'revenue, but only for returned lines' or 'revenue, but for last year'.

```
Returned Lines = CALCULATE ( COUNTROWS ( Fact_Sales ), Fact_Sales[is_returned] = 1 )
Cancelled Orders = CALCULATE ( COUNTROWS ( Fact_Orders ), Fact_Orders[order_status] =
    "Cancelled" )
Revenue PY = CALCULATE ( [Net Revenue], SAMEPERIODLASTYEAR ( Dim_Date[Date] ) )
```

Removing filters: ALL, REMOVEFILTERS, ALLSELECTED

Function	Effect	Use
`ALL(table/column)` / `REMOVEFILTERS()`	Ignore filters on that table/column	Grand totals as denominator
`ALLSELECTED()`	Ignore the visual's row/column filters but keep slicers the user chose	'% of what the user is looking at'
`ALLEXCEPT`	Remove all filters except named columns	Keep one grouping

```
Revenue % of Total = DIVIDE ( [Net Revenue], CALCULATE ( [Net Revenue], ALLSELECTED () )
     )
```

Iterators (the X functions) and context transition

`SUMX`, `AVERAGEX`, `COUNTX`, `MAXX` walk through a table row by row, evaluate an expression in each row's context, then aggregate. **Context transition** is the rule that when a **measure** (or CALCULATE) is evaluated inside a row context, the current row becomes a filter. That is how `ADDCOLUMNS(VALUES(Product[name]), "@rev", [Net Revenue])` computes each product's revenue.

> **ANALOGY**
>
> Row context is you pointing at one student in a class register. Context transition is turning that finger-point into a filter ('only this student') so a measure computes that student's marks total.

Variables, DIVIDE, SWITCH, SELECTEDVALUE

```
Revenue YoY % =
VAR PY = [Revenue PY]                                          -- computed once, reused,
    readable
RETURN IF ( [Is Period In Data] = 1 && NOT ISBLANK ( PY ),
            DIVIDE ( [Net Revenue] - PY, PY ) )                -- DIVIDE avoids #DIV/0

Selected Metric = SWITCH ( SELECTEDVALUE ( 'Metric Selector'[Metric], "Net Revenue" ),
                  "Net Revenue", [Net Revenue], "Gross Profit", [Gross Profit],
                      "Orders", [Orders], ... )
```

- **VAR** stores a value once (faster, clearer, easier to debug).
- **DIVIDE(a, b)** returns blank instead of an error when b = 0. Always use it for ratios.
- **SWITCH(TRUE(), cond1, result1, ...)** is a readable multi-way IF.

- **SELECTEDVALUE(col, default)** returns the single selected slicer value (used for parameters and dynamic titles).

> **COMMON MISTAKE**
>
> **Measures cannot appear inside a boolean filter shortcut** like `Fact[x] > [Measure]` (error). Store the measure in a `VAR` first, then compare. We do this in the Seasonality Index measure (`VAR LastD = [Last Data Date]`).

Time intelligence

Functions such as `SAMEPERIODLASTYEAR`, `DATEADD`, `TOTALYTD`, `DATESINPERIOD` shift the date filter. They **require a continuous Date table marked as the date table** (no gaps, one row per day).

Measure	DAX idea	Answers
Revenue PY	`SAMEPERIODLASTYEAR`	Same period last year (for YoY)
MoM %	`DATEADD(date, -1, MONTH)`	Month over month growth
Revenue YTD	`TOTALYTD`	Year-to-date cumulative
Revenue FYTD	`TOTALYTD(..., "3/31")`	Fiscal (Apr-Mar) year-to-date
Rolling 3M	`DATESINPERIOD(date, MAX(date), -3, MONTH)`	Trailing 3-month total

> **IN THIS PROJECT**
>
> Our calendar runs to Dec-2026 but data ends Dec-2025. Without a guard, 2026 months show nonsense growth. `[Last Data Date]` and `[Is Period In Data]` blank out future periods: a detail that distinguishes experienced developers.

16.6 The project's measure library, by family

Family	Key measures	Purpose / DAX idea
Sales & profit	Net Revenue, Kept Revenue, Gross Profit, Gross Margin %, AOV	Single shared definitions; `SUM`, `DISTINCTCOUNT`, `DIVIDE`
Returns & ops	Return Rate %, Cancellation Rate %, Late Delivery %, Top Return Reason	`CALCULATE` with filters; two grains (lines vs orders)
Time	YoY %, MoM %, YTD, FYTD, Rolling 3M, Seasonality Index	Time intelligence + future-period guard
Ranking / Pareto	Product Rank, Cumulative Revenue %, ABC Class, Top N	`RANKX`, `SUMX(FILTER())`, what-if parameter
Customers	New / Returning Customers, Repeat Rate, Lapsed Customers, Top-10% Share	Context transition, virtual tables
Cohort	Cohort Size, Cohort Retention %	Disconnected table + `EDATE`; blank for unobserved months
Marketing	CAC, LTV 12M, LTV to CAC, Payback, Channel Health	Conformed channel dimension; calculated column for LTV
UX	Selected Metric, YoY Arrow, Dynamic Title	`SWITCH`, colours for conditional formatting
What-if	Net Revenue @ Cap, Profit Uplift	`SUMX` re-pricing with `MIN`, parameter slicer

Walkthrough 1: Cumulative Revenue % (the Pareto line)

```
Cumulative Revenue % =
VAR Cur = [Net Revenue]                                        -- this product's revenue
VAR Tbl = ALLSELECTED ( Dim_Product[product_name] )            -- all products currently
    selected
VAR Cum = SUMX ( FILTER ( Tbl, [Net Revenue] >= Cur ), [Net Revenue] )   -- add up
    everyone at least as big as me
VAR Tot = SUMX ( Tbl, [Net Revenue] )
RETURN IF ( ISBLANK ( Cur ), BLANK (), DIVIDE ( Cum, Tot ) )
```

For each product, sum the revenue of every product whose revenue is ≥ its own: that is the running total from the top, without needing to sort. Dividing by the total gives the cumulative share. `ABC Class` then reads this value (≤ 70% → A, ≤ 90% → B, else C).

Walkthrough 2: Cohort Retention % (matrix of cohort × month offset)

```
Cohort Retention % =
VAR C   = MAX ( Dim_Customer[Cohort Month] )                   -- the row's cohort
VAR Off = SELECTEDVALUE ( Offsets[Offset] )                    -- the column's month
    offset (0..12)
RETURN IF ( EDATE ( C, Off ) > [Last Data Month], BLANK (),    -- not yet happened:
    blank, not zero
            DIVIDE ( [Cohort Active Customers], [Cohort Size] ) )
```

`Offsets` is a **disconnected table** (no relationship) used only to supply column headers 0-12. `EDATE(C, Off)` moves the cohort month forward by the offset; `[Cohort Active Customers]` counts distinct customers who bought in that month. This reproduces the Python/SQL cohort table inside Power BI.

Walkthrough 3: RFM as calculated columns

Each customer row needs R, F, M values. Because the row becomes a filter on `Fact_Sales` (context transition), `Lifetime Orders = CALCULATE(DISTINCTCOUNT(Fact_Sales[order_id]))` returns **that customer's** orders. Scores are quintiles built from a rank (`RANKX`-style count of lower-ranked customers) over a unique key (value + customer number) so ties break deterministically, and `SWITCH(TRUE())` assigns the same eight segment names as Python and SQL. Because these are **columns**, 'RFM Segment' can be placed on an axis or slicer, which a measure cannot.

Walkthrough 4: LTV:CAC

```
GP 12M (column on Dim_Customer) =
VAR SignupDate = Dim_Customer[signup_date]
RETURN CALCULATE ( SUM ( Fact_Sales[gross_profit] ), Fact_Sales[order_date] <=
    SignupDate + 365 )

LTV 12M Gross Profit = CALCULATE ( AVERAGE ( Dim_Customer[GP 12M] ), Dim_Customer[Is
    Mature 12M] = TRUE () )
CAC       = DIVIDE ( [Marketing Spend], [New Customers] )
LTV to CAC = DIVIDE ( [LTV 12M Gross Profit], [CAC] )
```

The column stores each customer's first-year profit; the measure averages it over customers with a full year of history; spend comes from a different fact table through the shared `Dim_Channel`. That is a good example of **one visual combining two fact tables via a conformed dimension**.

16.7 DAX queries (DAX Query View)

Besides measures you can run **queries** with `EVALUATE`, which return a table: perfect for **testing** measures against SQL.

```
EVALUATE
SUMMARIZECOLUMNS ( Dim_Date[Year], "Net Revenue", [Net Revenue], "Gross Profit", [Gross
    Profit], "Orders", [Orders] )
ORDER BY Dim_Date[Year]

DEFINE MEASURE _Measures[Net Revenue @10% Cap] =
       SUMX ( Fact_Sales, Fact_Sales[gross_sales] * ( 1 - MIN (
           Fact_Sales[discount_pct], 10 ) / 100 ) )
EVALUATE SUMMARIZECOLUMNS ( Dim_Product[category], "Revenue @ cap", [Net Revenue @10%
    Cap] )
```

`DEFINE MEASURE` *creates a temporary measure for the query only: handy for what-if tests without changing the model.*

Expected checks (should match SQL exactly): **2023** net revenue 38,809,309 · **2024** 88,935,118 · **2025** 133,807,961; 2025 gross profit 39,524,593; all-time orders 27,871; cancellation rate 4.25%.

16.8 Designing the dashboard

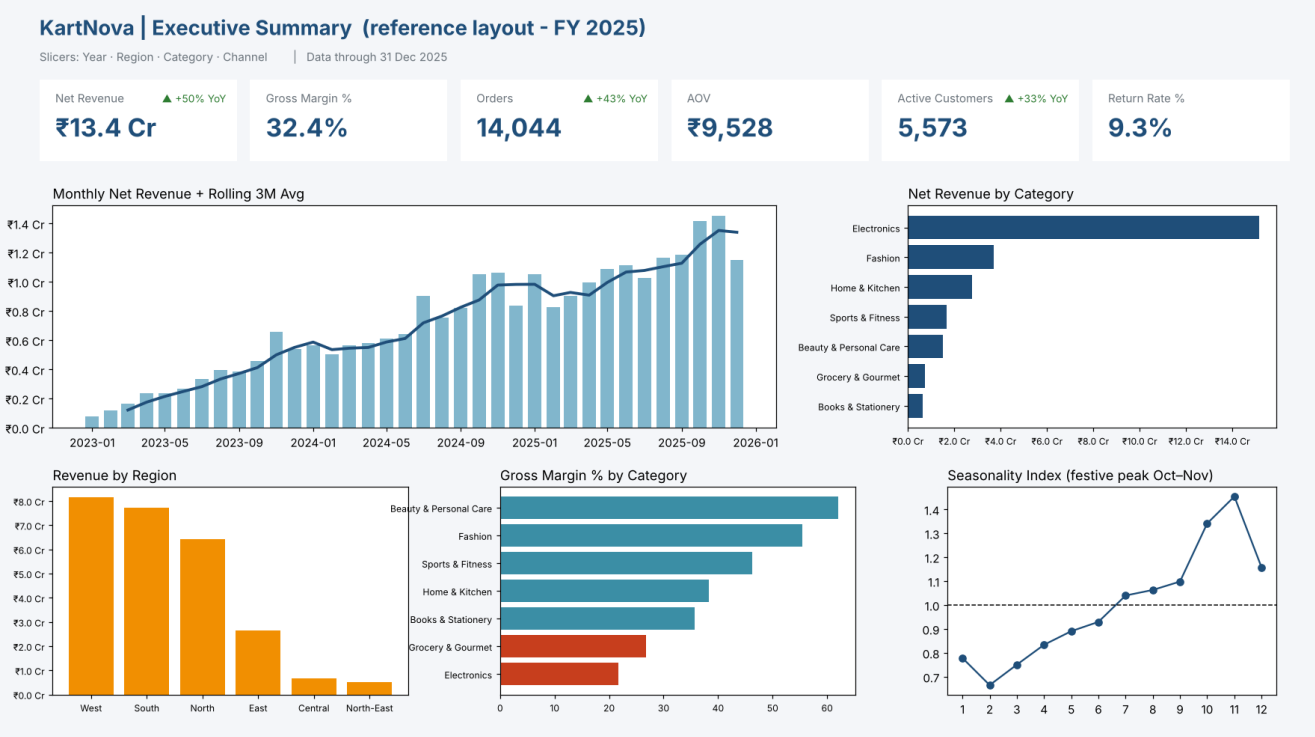


Figure 16.3 - Reference layout of page 1 (rendered in Python from the real data; the .pbix is built by following the blueprint).

Page	Business question	Main visuals
1 Executive Summary	Are we growing profitably?	6 KPI cards with YoY arrows, trend with metric switcher, category and region bars, seasonality
2 Sales & Product	Where do money and profit come from?	Treemap, Pareto combo, ABC table, margin vs returns scatter
3 Customers	Who are best/at-risk customers?	RFM segment bars, cohort matrix, new vs returning
4 Marketing ROI	Which channels deserve budget?	CAC, LTV:CAC table with status, CAC vs LTV scatter
5 Operations & Returns	Where do we lose margin after the sale?	Region SLA heatmap, cancellation by payment, return reasons
6 Pricing What-If	What if we cap discounts?	Discount-cap slicer, uplift cards, waterfall

Design principles to quote

- **One page = one business question.** 5-8 visuals, most important top-left.
- KPI cards on top, trend in the middle, drill-downs below. Keep a consistent colour meaning (green good, red bad) and never use colour alone (add arrows/text).
- Use **slicers sparingly** (Year, Region, Category, Channel) and sync them across pages; add 'reset' bookmarks.
- **Dynamic titles** and a 'data through' footer build trust. Add **tooltips** and **drill-through** (customer detail) for depth.
- Test on **mobile layout**; check contrast and alt text for accessibility.

16.9 Governance and performance

- **Row-Level Security (RLS):** a role defined by a DAX filter such as `[region] = "West"` restricts what each person sees; production uses a mapping table and `USERPRINCIPALNAME()`.
- **Refresh:** import mode with scheduled refresh; **Incremental Refresh** loads only new partitions for large facts; a gateway for on-premises sources.
- **Import vs DirectQuery:** Import = fast, in-memory; DirectQuery = live but slower and limited DAX. Composite models mix both.
- **Performance tuning:** star schema, remove unused columns, avoid bi-directional filters and heavy calculated columns, use `VAR`, check with **Performance Analyzer** (target < 1 second per visual).
- **Calculation groups** (Tabular Editor) can replace a dozen time-intelligence measures with one reusable pattern.

16.10 Common DAX errors and fixes

Symptom	Likely cause	Fix
Total ≠ sum of rows	Measure is not additive (ratios, distinct counts)	Expected; or rewrite with SUMX over the row level
Wrong number after adding a slicer	Relationship direction/inactive relationship or filter not propagating	Check model view, filter direction, USERELATIONSHIP
Time intelligence returns blank	Date table not marked / has gaps / wrong column used	Mark date table; use Date column from the Date table
Circular dependency	Calculated columns referring to each other	Move logic to Power Query or a measure
Slow visual	Iterator over huge table, bi-directional filter	Pre-aggregate, simplify, test in Performance Analyzer

INTERVIEW ANGLE

'Measure vs calculated column?' See table in 16.5: measures react to filters and are computed on demand; columns are row-level, stored, usable as axes.

'Explain filter context vs row context.' Filter context = which rows a calculation sees (from slicers/visual); row context = the current row during iteration. CALCULATE can convert row → filter (context transition).

'Why a star schema?' Simple, fast, unambiguous filter propagation; matches how people slice data; engine is optimised for it.

'Your KPI differs from Finance. What do you do?' Compare definitions, filters, grain, date logic; reconcile step by step; document the agreed rule. We reconcile Python = SQL = DAX in this project.

'How would you show the same chart for Revenue, Orders or Profit?' Disconnected 'metric selector' table + `SWITCH(SELECTEDVALUE())` measure (or field parameters).

17. Insights and recommendations

Analysis ends with a **decision**. For every finding, a senior analyst states: the evidence, the recommendation, the estimated impact, and the caveat. Memorise this format; it is how you present in interviews and to leadership.

#	Finding (evidence)	Recommendation	Impact / caveat
1	Discount depth does not raise units (ρ = 0.004). Margin falls 38.5% → 24.4% from no-discount to the 11-20% band; > 20% discounts raise returns to 12.9%	Cap line discounts at 10% outside festive peaks; A/B test the cap	≈ +₹83.9 L profit (+10.9%); upper bound, assumes zero volume loss
2	Electronics = 58% of revenue but 22% margin; Fashion margin 55% but 19% returns	Bundle accessories with laptops/phones (lift 13×); size guides and fit content for Fashion	Margin mix improves; each 1pp fewer Fashion returns ≈ ₹3.7 L
3	LTV:CAC: Referral 12.5×, Organic 11.2× … Paid Search 2.5× (payback 4.8 months)	Scale referral; cap Paid Search CAC; reallocate part of its ₹60 L three-year spend	More profit per marketing rupee; attribution caveat
4	Champions = 17% of customers / 43% of revenue; 'Can't Lose Them' (645 customers, ₹3.7 Cr lifetime) silent ≈17 months	VIP programme; personal win-back for Can't Lose Them; stop discounting 'Lost'	Protects ≈ ₹14 Cr revenue base
5	Churn model AUC 0.83; top 4 deciles capture 54% of churners; 3,617 'High' risk customers hold ₹5.8 Cr	Trigger win-back at ~1.5× a customer's usual gap; rank by revenue at risk	Focus on ~half the base; validate with an A/B test
6	Retention 18.8% / 16.3% / 13.8% at M1/M3/M6; 62% place a 2nd order after ≈95 days	Post-purchase journeys at day 30/60/90	+1pp M1 retention compounds across cohorts
7	COD cancels 8.9% vs 3.3% prepaid ($p \approx 10^{-73}$)	Prepaid incentive; confirmation step for high-value COD	≈ 5pp of COD orders saved
8	North-East avg 7.7 days, 54% > 7 days; East 6.3 days	Regional fulfilment partner / forward stocking	Lower cancellations and returns
9	Festive peak Oct-Nov (≈1.4-1.6× average) but AOV ₹529 lower in festive months	Pre-build A-class stock by September; targeted not blanket discounts	Fewer stock-outs; protects margin
10	Log-trend forecast (backtest MAPE 14.5%) projects ≈ ₹1.0-1.3 Cr per month in H1-2026	Planning baseline for inventory/cash; re-fit monthly	80% band ≈ −16% / +19%

17.1 Caveats to volunteer

- **Synthetic data:** patterns were designed; real data is noisier and needs more cleaning.
- **What-if assumptions:** the discount cap assumes no volume response; must be experimentally validated.
- **LTV** is a 12-month gross-profit proxy, not a probabilistic lifetime model; **CAC** is channel-level and attribution is simplified.
- **Seasonality index** is growth-confounded; a decomposition (STL) is the proper next step.
- **Churn** label is a business definition (180 days); a production version also validates on a later time period and checks calibration.
- **Boosting did not beat logistic regression:** the signal is dominated by recency, so simplicity wins.

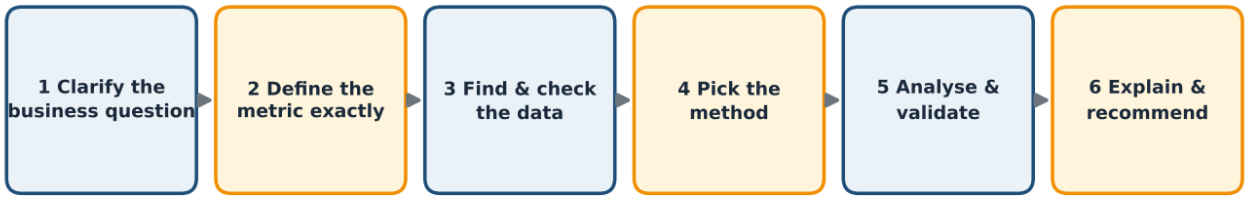
INTERVIEW ANGLE

'What are the limitations of your project?' is a **gift**. Naming 3-4 real limitations calmly (synthetic data, assumptions, proxies) signals maturity far more than claiming perfection.

18. The scenario playbook: answering questions you have never seen

Interviewers rarely ask exactly what is in your project. They give **new scenarios** to see how you think. This chapter gives you a reusable method, a map from problem types to techniques, and fourteen worked scenarios. Practise saying them aloud.

18.1 The universal method (use it out loud)



Always ask: 'So what? What decision will change because of this number?'

Figure 18.1 - The six steps. State them as you go; interviewers grade your structure.

Step	What you say	Example phrase
1 Clarify	Goal, decision, audience, time frame, constraints	'Before analysing, who is the decision-maker and what would they do differently with the answer?'
2 Define	Exact metric, numerator/denominator, grain, period	'I will define churn as no purchase in 180 days. Is that consistent with the business?'
3 Data	Sources, quality checks, joins, known gaps	'I would check duplicates, nulls, reconcile totals to Finance, and confirm the grain of each table.'
4 Method	Simplest approach first, then escalate	'Start with a segmented comparison; if prediction is needed, a baseline model.'
5 Validate	Sanity checks, confounders, uncertainty	'I would test significance and effect size, and check that a hidden factor like seasonality is not driving it.'
6 Recommend	Answer first, 2-3 numbers, action, risk, next step	'Recommendation: cap discounts at 10%; expected +₹84 L; validate with an A/B test.'

18.2 Decomposition trees: break any metric into drivers

When a metric moves, **decompose** it. Each branch is something you can measure and someone can act on.

Metric	Decomposition	What to check when it falls
Revenue	Customers × orders per customer × average order value (AOV); AOV = items per order × price per item	Fewer customers (acquisition/retention)? Fewer orders each? Smaller baskets? Lower prices/more discounts?
Profit	Revenue − COGS − discounts − returns − fulfilment − marketing	Mix shift to low-margin products? Higher returns? Discount creep? Cost inflation?

Metric	Decomposition	What to check when it falls
Conversion	Visitors → product view → add to cart → checkout → payment → success	Which step's drop-off changed? By device/channel?
Churn	Customers at start − customers lost + customers added	Which segment/tenure/channel churns? Service failures? Price changes?
Delivery time	Order → pack → ship → transit → last mile	Which leg slowed? Which region/carrier/warehouse?

WORKED EXAMPLE

Revenue = customers × orders per customer × AOV. KartNova 2025: 5,573 buyers × 2.52 orders each × ₹9,528 AOV ≈ ₹13.4 Cr. If revenue falls 10%, you test each factor: did buyers fall (acquisition), orders per buyer (retention), or AOV (pricing/mix)?

18.3 Which technique for which problem?

Question type	Technique	In this project	Typical tools
What is the state of the business?	KPIs, trends, YoY	Scorecard, monthly trend	SQL, Power BI
Where is value/loss concentrated?	Group-by, Pareto, ABC	Category economics, Pareto	SQL, pandas
Who are our customer types?	RFM, K-Means	Chapter 8	pandas, sklearn, SQL NTILE
Do customers stay?	Cohort retention, survival	Chapter 9	SQL, pandas
Who will leave / default / buy?	Classification (logistic, trees)	Churn model	sklearn
Is this difference real?	Hypothesis test, confidence interval	Chapter 13	scipy
Did the change cause the effect?	A/B test, difference-in-differences	Discount cap (proposed)	experiments
What will happen next?	Time-series forecasting	Chapter 14	regression, ETS, Prophet
What goes with what?	Association rules	Basket analysis	pandas, SQL self-join
Where to spend the budget?	Unit economics (CAC/LTV), marginal ROI	Channel table	SQL, Excel
Is the data right?	Profiling, constraints, reconciliation	Chapter 6	SQL, pandas

18.4 The same toolkit in other industries

Industry	Churn means...	'Revenue per customer' becomes...	Cohorts / RFM become...	Special twist
Telecom / SaaS	Cancelling a subscription (contractual: you see it)	ARPU / MRR	Signup-month cohorts; usage-based RFM	Survival analysis; contract end dates
Banking	Closing an account / stopping use of card	Net interest + fees per customer	Product holding, balance, transaction frequency	Credit risk, fraud; regulation and fairness
Food delivery	No order in 30 days	Orders per month × AOV	First-order cohorts by city; order frequency	Delivery time and restaurant ratings drive retention

Industry	Churn means...	'Revenue per customer' becomes...	Cohorts / RFM become...	Special twist
Healthcare	Missed follow-ups / readmission	Cost per patient	Admission cohorts; risk strata	Privacy; class imbalance; clinical validation
HR	Employee attrition	Cost per employee	Hire-date cohorts; tenure bands	Bias and ethics; small samples
Logistics	Customer switching carriers	Revenue per shipment	Lane/region cohorts	SLA and cost-to-serve metrics
Ad-tech / media	Users inactive	RPM / ARPU	Install-date cohorts; DAU/MAU	Experiments, attribution

KEY IDEA

When you meet an unfamiliar industry, **translate the vocabulary, not the method**. Ask: What is a 'customer'? What is a 'purchase'? What is 'churn'? What is the money metric? Once you map those four words, RFM, cohorts, churn models, funnels and forecasting all transfer.

18.5 A/B testing in five minutes

An **A/B test** is a randomised experiment: randomly split users into **control (A)** and **treatment (B)**, change one thing for B, and compare a metric. Randomisation makes the groups comparable, so a difference can be attributed to the change (**causal**), which observational analysis cannot guarantee.

1. **Hypothesis & metric:** 'Capping discounts at 10% increases profit per customer'. Choose one **primary metric** plus **guardrails** (conversion, returns, NPS).
2. **Unit of randomisation:** customer, session, store or region (avoid contamination).
3. **Sample size & power:** decide the smallest effect worth detecting (MDE), significance level (5%) and power (80%); compute required users **before** starting.
4. **Run for full cycles** (whole weeks; avoid holidays) and **do not peek** and stop when it first looks significant (inflates false positives).
5. **Analyse:** test the difference, give the effect size and confidence interval; check **sample ratio mismatch** (are groups the expected sizes?).
6. **Decide:** ship if significant **and** practically valuable and guardrails are fine.

COMMON MISTAKE

Common experiment traps: peeking, multiple metrics ('p-hacking'), novelty effects, interference between groups, tiny samples, stopping early, and confusing statistical with business significance.

18.6 Fourteen worked scenarios

Scenario 1: 'Revenue dropped 20% last month. What do you do?'

Situation: Month-over-month fall in a retailer's net revenue.

Ask first: Is it real (data issue, late-loading data, a definition change)? Compared with what (last month, last year)? Which scope (all products, regions)?

- **Verify the data first**: freshness, duplicates, missing days, reconcile with Finance.
- **Seasonality check**: compare with the same month last year (YoY), not only MoM.
- **Decompose**: customers × orders/customer × AOV; then slice by category, region, channel, device, new vs returning customers.

- **Find the leg that moved**; look for events: stock-outs, price changes, campaign end, outage, competitor.
- **Recommend** a fix and an owner, with a monitoring alert.

> **COMMON MISTAKE**
>
> Jumping to conclusions ('marketing failed') before verifying data and seasonality.

Scenario 2: 'Build a churn solution for a subscription (telecom/SaaS) company.'

Situation: Customers pay monthly; leadership wants to reduce cancellations.

Ask first: Churn definition (cancelled vs inactive)? Horizon (next 30/60 days)? Cost of an offer vs value of a saved customer?

- **Contractual churn** is observed directly (cancel date), unlike our non-contractual retail case where silence defines churn. Use **survival analysis** or a classifier for 'cancel in the next 30 days'.
- **Features** from before the snapshot: tenure, usage trend, support tickets, billing issues, plan, discounts expiring, NPS.
- **Temporal split** to avoid leakage; baseline logistic vs tree model; judge by PR-AUC, lift, calibration.
- **Act**: rank by (probability × value × likelihood the offer works), run an **A/B test** of the retention offer, measure net profit (offer cost included).

Scenario 3: 'Did our email campaign work?'

Situation: Marketing emailed 200,000 customers and sales rose.

Ask first: Was there a random holdout? How were recipients chosen? What window and metric?

- If a **randomised holdout** exists: compare conversion/revenue per user between sent and holdout, with a confidence interval (incremental lift).
- If **no holdout**: beware **selection bias** (they emailed engaged customers) and **timing** (seasonality). Use a matched comparison or difference-in-differences, state limits.
- Compute **incremental profit** = lift × customers × margin − campaign cost. Report ROI, not just clicks.

> **COMMON MISTAKE**
>
> Crediting the campaign for sales that would have happened anyway (the baseline problem).

Scenario 4: 'Which products should we discount?'

Situation: Pricing manager wants a promotion list.

Ask first: Goal: volume, clearing stock, or profit? Constraints: margin floor, stock levels?

- Compute **margin, price elasticity proxy and stock cover** per product.
- Our analysis showed discount depth did not raise units and erodes margin, so default to **targeted, shallow** offers on slow-moving high-margin items; avoid thin-margin items (Electronics).
- Design a **test** with control groups; measure incremental profit, not revenue.

Scenario 5: 'Customers complain about late delivery. Investigate.'

Situation: Complaints are rising.

Ask first: Define 'late' (promised vs actual)? Which geography/period?

- Compute **delivery days and late %** by region, warehouse, carrier, order week; test differences (Kruskal-Wallis).

- Break the journey into legs (pack, ship, last mile) to find the slow leg.
- Quantify impact: cancellations, returns and repeat purchase for late vs on-time orders.
- Recommend a carrier/stocking change; track SLA weekly (our NE example: 54% > 7 days).

Scenario 6: 'Segment the customers of a bank.'

Situation: Bank wants targeted offers.

Ask first: Objective (cross-sell, retention, risk)? Available data (balances, transactions, products)?

- Replace RFM with **behavioural features**: balance, transaction frequency/recency, products held, channel usage, tenure.
- Standardise, log-transform skewed fields, run **K-Means**, choose k by silhouette **and** usability.
- Profile clusters (size, value, behaviour), name them, map each to an action; validate stability over time.

Scenario 7: 'Forecast demand for the next quarter for inventory.'

Situation: Planner needs SKU-level numbers.

Ask first: Granularity (SKU/store/week)? Promotions known? Lead times?

- Start with the **naive and seasonal-naive** benchmarks; then regression/ETS; add drivers (price, promotions, holidays).
- **Backtest** with rolling origins; report MAPE/WAPE and bias; produce **prediction intervals**.
- Translate into **safety stock** from forecast uncertainty and lead time; hierarchical reconciliation (SKU sums to category).

Scenario 8: 'Design a dashboard for the head of sales.'

Situation: A new dashboard request.

Ask first: What decisions does she make weekly/monthly? Which KPIs does she already track? Who else uses it?

- **Requirements first**: 5-7 KPIs, definitions agreed, drill paths, refresh frequency.
- **Model**: star schema; measures for KPIs and YoY/MTD/YTD; RLS if regional.
- **Layout**: KPI cards, trend, breakdowns, exceptions; one question per page.
- **Test with users**, reconcile numbers to Finance, document definitions.

Scenario 9: 'The dashboard number does not match Finance. Debug it.'

Situation: Revenue differs by 3%.

Ask first: Which period, filters, definition? Which one is right?

- Compare **definitions** (gross vs net, returns, tax, cancelled orders, timezone/date cut-off).
- **Reconcile top-down** by month, then by region/category until the gap localises.
- Check **joins** (fan-out duplicates), **filters**, **NULL handling**, late-arriving data.
- Fix, document the agreed rule, add an automated reconciliation test (as we do between SQL, Python and DAX).

Scenario 10: 'Detect fraudulent or anomalous transactions.'

Situation: Payments team wants alerts.

Ask first: Labelled fraud history? Cost of false positives vs missed fraud?

- Without labels: **rules** (velocity, geo mismatch, amount outliers) and **anomaly detection** (z-scores, isolation forest). With labels: classifier with heavy **class imbalance** handling.
- Judge by **precision@k**, recall, PR-AUC, and monetary cost; keep a human review loop; monitor drift.

Scenario 11: 'Why are employees leaving?'

Situation: HR attrition is 22%.

Ask first: Voluntary or involuntary? Which teams?

- Cohort by hire date, survival curves by team/manager/compensation band; logistic model for drivers; compare leavers vs stayers.
- Be careful with **bias, privacy and small samples**; present drivers as hypotheses to test, not verdicts.

Scenario 12: 'Does our loyalty programme increase spend?'

Situation: Members spend 30% more than non-members.

Ask first: How did customers join (self-selection)?

- Naive comparison is **biased**: engaged customers join loyalty schemes. Use a **pre/post with matched controls**, difference-in-differences, or a staggered rollout experiment.
- Report incremental profit net of reward cost.

> **COMMON MISTAKE**
>
> Selection bias: correlation is not causation.

Scenario 13: 'App sign-ups are the same but purchases fell. Why?'

Situation: Funnel conversion decreased.

Ask first: Which device/platform/version/channel? When did it start?

- Build the **funnel** (sign-up → browse → cart → pay → success) and compare periods; localise the step and segment.
- Check releases, payment gateway errors, pricing or stock changes, traffic quality (new low-intent channel).
- Use a **Simpson's paradox** check: overall rate can fall because of mix shift even if each segment improved.

Scenario 14: 'Choose a metric/model for rare-event prediction (credit default 2%).'

Situation: Only 2% of loans default.

Ask first: Costs of false positives vs false negatives?

- Accuracy is useless (98% by predicting no default). Use **PR-AUC, recall at a precision floor, KS, expected cost**.
- Handle imbalance with class weights/resampling; use stratified, time-based validation; calibrate probabilities; set the threshold by cost.

> **INTERVIEW ANGLE**
>
> **Pattern across all 14:** (1) clarify and define, (2) check data/baseline, (3) decompose, (4) simplest valid method, (5) beware bias/confounding/leakage/censoring, (6) quantify impact and propose a test. If you only remember one slide of this guide, remember that list.

19. Interview question bank with model answers

Answers are deliberately short (what you would say in 30-60 seconds). Practise them aloud; then add a project example to each.

19.1 About the project

Q. Give a 60-second summary of your project.

I built an end-to-end analytics project on a simulated Indian e-commerce retailer: 9,000 customers, 29,000 orders, three years. I cleaned messy raw data with a documented defect log, then analysed the same clean dataset three ways: Python (RFM, cohorts, churn model, statistical tests, forecasting), SQL (22 window-function queries and integrity checks) and Power BI/DAX (a star-schema model with about 80 measures). I reconciled numbers across all three. Key findings: discounts didn't raise volume but cut margin (a 10% cap is worth about ₹84 lakh), Paid Search returns only 2.5× its acquisition cost vs 12.5× for Referral, and the top 10% of customers produce 49% of revenue.

Q. Why did you use synthetic data?

Real retailer data is confidential. I generated data with realistic dynamics (seasonality, churn, margins, returns, injected defects) so the full workflow could be demonstrated and reproduced. I'm transparent that findings are properties of the simulation; the methods transfer unchanged to real data.

Q. What was the hardest part?

Keeping one definition of revenue and profit across Python, SQL and DAX. I defined the rules once (net revenue, refunds, COGS reversal), implemented them identically, and wrote reconciliation checks. For example 2025 net revenue matches to the rupee in Python and SQL.

Q. What would you do differently with real data?

Validate churn on a later time period and calibrate probabilities, use a proper multi-touch attribution or incrementality test for channel ROI, decompose seasonality properly (STL), and run an A/B test before any discount-cap rollout.

Q. What was your most valuable insight?

That discount depth had no relationship with units per line (Spearman $\rho \approx 0$) yet cut margin from 38% to 24%, so capping it frees profit. It is valuable because it turns an analysis into a priced decision, with the caveat that it must be validated by an experiment.

Q. How did you validate your results?

Cross-tool reconciliation (Python = SQL = DAX queries), SQL integrity assertions, backtesting for the forecast, a held-out test set and lift for the churn model, and significance plus effect sizes for comparisons.

Q. Which finding would you present first to the CEO and why?

Profit leakage: deep discounts and Fashion returns, plus the channel reallocation, because they have quantified rupee impact and clear owners. Retention insights come next as the growth ceiling.

19.2 Python and pandas

Q. `merge` vs `join` vs `concat`?

`merge` joins on columns/keys (SQL-style); `join` joins on the index; `concat` stacks tables vertically or horizontally without matching keys.

Q. How do you find and remove duplicates?

`df.duplicated(subset=key).sum()` to count; `df.drop_duplicates(subset=key)` to remove. Always check why duplicates exist and count rows before and after.

Q. `apply` vs vectorised operations?

Vectorised column operations run in optimised C and are much faster; `apply` runs Python per row/group and is slow. Prefer vectorised math, `np.where`, and `groupby.transform`.

Q. What does `groupby().transform()` do versus `agg()`?

`agg` returns one row per group; `transform` returns a result the same length as the original, repeating each group's value on every row (e.g., fill missing age with group median).

Q. How do you handle a dataset too big for memory?

Read in chunks, select needed columns and smaller dtypes (`category`, `int32`), filter early, aggregate before joining, or use Parquet/DuckDB/Spark.

Q. How do you avoid a many-to-many join explosion?

Check key uniqueness (`df.key.is_unique`), validate with `merge(..., validate='m:1')`, and compare row counts after the join.

Q. What is the difference between `loc` and `iloc`?

`loc` selects by label/boolean mask; `iloc` selects by integer position.

Q. Why `qcut` with `rank(method='first')`?

`qcut` fails or produces uneven bins when many values tie (e.g., most customers have 1 order). Ranking first gives unique values so equal-sized quantiles can be formed.

Q. How do you make an analysis reproducible?

Scripts not notebooks-only, fixed random seeds, a pinned requirements file, relative paths, a single entry point (`run_all.py`) and logged outputs.

19.3 SQL

Q. WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q. INNER vs LEFT JOIN, and when can a join change row counts?

INNER keeps matches only; LEFT keeps all left rows. Both can multiply rows if the right table has multiple matches per key.

Q. RANK vs DENSE_RANK vs ROW_NUMBER?

On ties: ROW_NUMBER gives unique numbers, RANK leaves gaps after ties (1,1,3), DENSE_RANK doesn't (1,1,2).

Q. Write the top 3 products per category.

`WITH r AS (SELECT category, product, SUM(rev) rev, ROW_NUMBER() OVER (PARTITION BY category ORDER BY SUM(rev) DESC) rn FROM t GROUP BY category, product) SELECT * FROM r WHERE rn <= 3;` (or aggregate in one CTE then rank in the next).

Q. How do you compute month-over-month growth?

Aggregate to monthly totals in a CTE, then `revenue / LAG(revenue) OVER (ORDER BY month) - 1`.

Q. How do you find customers who never ordered?

`LEFT JOIN orders ... WHERE orders.customer_id IS NULL` or `NOT EXISTS`. Avoid `NOT IN` if the subquery can contain NULLs.

Q. What is a CTE and why use it?

A named temporary result defined with WITH. It makes multi-step logic readable and reusable within a query; recursive CTEs handle hierarchies.

Q. What is an index and when does it hurt?

A data structure that speeds up lookups and joins. It slows inserts/updates and uses space, so index columns used frequently in filters and joins.

Q. How would you detect a 'gap' or streak in dates?

Gaps-and-islands: `date - ROW_NUMBER() OVER (PARTITION BY user ORDER BY date)` is constant within a consecutive streak; group by it.

Q. Explain `COUNT(*)`, `COUNT(col)`, `COUNT(DISTINCT col)`.

All rows; non-NULL values of col; distinct non-NULL values.

Q. How do you debug a query whose total looks too high?

Check for join fan-out (duplicate keys), missing filters (status), and grain mismatch; compare running totals step by step through CTEs.

Q. What is normalisation vs a star schema?

Normalisation (3NF) reduces redundancy for transactional systems; a star schema denormalises into facts and dimensions for fast analytical queries.

19.4 Power BI and DAX

Q. Measure vs calculated column?

A measure is computed on demand in the filter context and reacts to slicers; a calculated column is computed per row at refresh and stored. Use columns for attributes/axes, measures for KPIs.

Q. Explain filter context and row context.

Filter context is the set of rows a calculation can see (from slicers/visual); row context is the current row during iteration. CALCULATE can turn row context into filter context (context transition).

Q. What does CALCULATE do?

Evaluates an expression after modifying the filter context: adding, replacing or removing filters. It is how you create 'revenue for returned lines' or 'last year revenue'.

Q. ALL vs ALLSELECTED?

ALL removes every filter on the table/column; ALLSELECTED removes visual-level filters but keeps slicer selections: ideal for '% of what the user is viewing'.

Q. Why do time-intelligence functions need a date table?

They shift a contiguous date column; gaps or a non-marked table breaks them. The Date table must have one row per day and be marked as the date table.

Q. Why DIVIDE instead of '/'?

DIVIDE handles divide-by-zero safely (returns blank or a chosen value) and is cleaner than IF checks.

Q. What is a disconnected table used for?

Slicer/parameter values without relationships (metric selector, what-if cap, cohort month offsets), read with SELECTEDVALUE.

Q. Import vs DirectQuery?

Import loads data into the in-memory model: fastest and full DAX. DirectQuery queries the source live: fresher but slower and with limitations.

Q. How do you improve a slow report?

Star schema, remove unused columns, avoid bi-directional filters and heavy calculated columns, use variables, reduce visuals per page, and use Performance Analyzer and DAX Studio.

Q. How would you implement row-level security?

Create roles with DAX filters on a dimension (e.g., `[region] = "West"`), or dynamic RLS with a user-to-region mapping table and USERPRINCIPALNAME(); test with 'View as'.

Q. Why two fact tables in your model?

Revenue is at delivered line grain, but cancellation and delivery metrics need all orders including cancelled ones at order grain. Mixing them would corrupt either metric.

19.5 Statistics

Q. Explain a p-value simply.

The probability of seeing a result at least this extreme if there were truly no effect. Small p means the result is unlikely to be just luck.

Q. Statistical vs practical significance?

With large samples, tiny differences become significant. Report effect size and confidence interval and judge by business impact.

Q. Type I vs Type II error?

Type I: false alarm (reject a true null). Type II: miss a real effect. Power = 1 − Type II probability.

Q. Mean vs median; when each?

Median for skewed data (revenue, delivery days); mean when roughly symmetric or when totals matter.

Q. What is a confidence interval?

A range computed so that, over repeated studies, 95% of such ranges contain the true value. It expresses precision.

Q. When do you use non-parametric tests?

Skewed data, outliers, ordinal data, small samples, when normality assumptions fail.

Q. What is correlation vs causation; how do you establish causation?

Correlation is co-movement; causation needs randomisation (A/B test) or credible quasi-experimental design controlling confounders.

Q. What is Simpson's paradox?

A trend that appears in subgroups can reverse in the aggregate due to mix differences; always check segments.

Q. How do you decide sample size for an A/B test?

From the baseline rate, minimum detectable effect, significance level and power; compute before starting.

Q. What is the bootstrap?

Resampling the data with replacement many times to estimate the sampling distribution and confidence intervals without distributional assumptions.

19.6 Machine learning

Q. What is data leakage?

Using information unavailable at prediction time (e.g., future data) in features, inflating test performance. Prevent with temporal splits and careful feature timing.

Q. Precision vs recall?

Precision: of predicted positives, how many are right. Recall: of actual positives, how many we catch. The trade-off is set by the cost of each error.

Q. What is ROC-AUC?

The probability that a random positive is ranked above a random negative; 0.5 is chance, 1 is perfect. It is threshold-independent but can be optimistic for rare classes (use PR-AUC).

Q. Overfitting: how to detect and reduce?

Large gap between train and test performance. Use hold-out/cross-validation, regularisation, simpler models, more data, early stopping.

Q. Logistic regression vs gradient boosting?

Logistic: linear, fast, explainable. Boosting: non-linear, higher capacity, less interpretable and more prone to overfit. Always benchmark with the simple model.

Q. How do you handle class imbalance?

Appropriate metrics (PR-AUC, recall at precision), class weights, resampling, threshold tuning by cost, stratified splits.

Q. How do you explain a model to business users?

Feature importance/SHAP/permutation importance in plain words, decile lift in business terms, and examples; and show the action the model enables.

Q. How do you choose k in K-Means?

Silhouette and elbow plots, stability across runs, and whether clusters are actionable.

19.7 Business and behavioural

Q. Tell me about a time data contradicted a stakeholder's belief.

STAR: Situation: a belief that deep discounts drive sales. Task: test it. Action: ran a rank-correlation and band analysis and showed no volume lift but a 14-point margin drop. Result: proposed a capped-discount test with ≈ ₹84 lakh upside (simulated project, explained as such).

Q. How do you prioritise multiple urgent requests?

Clarify the decision and deadline behind each, estimate impact and effort, align with the stakeholders openly, deliver the smallest valuable version first.

Q. How do you communicate with non-technical stakeholders?

Lead with the answer, use 2-3 numbers and a simple chart, translate jargon, state assumptions and what decision follows.

Q. What do you do when you find an error in a delivered report?

Tell stakeholders immediately, quantify the impact, fix and re-issue, add a test so it cannot recur.

Q. Why analytics, and what do you want to learn next?

I like turning ambiguous questions into decisions with numbers. Next: experimentation depth, causal inference and production data pipelines.

20. Cheat sheets

20.1 SQL

Task	Snippet
Rows per group	`SELECT g, COUNT(*) FROM t GROUP BY g;`
Filter groups	`... GROUP BY g HAVING COUNT(*) > 10`
% of total	`x * 1.0 / SUM(x) OVER ()`
Running total	`SUM(x) OVER (ORDER BY d)`
Previous value	`LAG(x, 1) OVER (PARTITION BY id ORDER BY d)`
Rank in group	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)`
Buckets	`NTILE(5) OVER (ORDER BY x)`
Conditional count	`SUM(CASE WHEN cond THEN 1 ELSE 0 END)`
Anti-join	`FROM a LEFT JOIN b USING(k) WHERE b.k IS NULL`
Safe divide	`x * 1.0 / NULLIF(y, 0)`
Dedupe	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY id) = 1`

20.2 pandas

Task	Snippet
Load	`pd.read_csv(p, parse_dates=[...])`
Filter	`df[(df.a > 5) & (df.b == 'x')]`
Group + aggregate	`df.groupby('g').agg(total=('x','sum'), n=('id','nunique'))`
Merge	`a.merge(b, on='k', how='left', validate='m:1')`
Missing	`df.isna().sum()`, `df.fillna(v)`, `df.dropna()`
Dates	`df.d.dt.to_period('M')`, `(d2 - d1).dt.days`
Quantiles	`pd.qcut(df.x, 5, labels=False)`
Pivot	`df.pivot_table(index, columns, values, aggfunc)`
Rate from 0/1	`df.flag.mean()`
Group-wise fill	`df.x.fillna(df.groupby('g').x.transform('median'))`
Cumulative	`df.x.cumsum()`, `df.x.pct_change()`, `df.x.rolling(3).mean()`

20.3 DAX

Task	Pattern
Sum / count	`SUM(T[x])`, `COUNTROWS(T)`, `DISTINCTCOUNT(T[id])`
Change filter	`CALCULATE([M], T[col] = "v")`
Ignore filters	`CALCULATE([M], REMOVEFILTERS(T))`
% of total	`DIVIDE([M], CALCULATE([M], ALLSELECTED()))`
Same period last year	`CALCULATE([M], SAMEPERIODLASTYEAR(D[Date]))`
YTD / fiscal YTD	`TOTALYTD([M], D[Date])`, `TOTALYTD([M], D[Date], "3/31")`
Rolling 3M	`CALCULATE([M], DATESINPERIOD(D[Date], MAX(D[Date]), -3, MONTH))`
Rank	`RANKX(ALLSELECTED(P[name]), [M], , DESC)`

Task	Pattern
Row-by-row math	`SUMX(T, T[qty] * T[price])`
Selected parameter	`SELECTEDVALUE(Param[Col], default)`
Multi-way if	`SWITCH(TRUE(), c1, r1, c2, r2, else)`

20.4 Statistics and metrics formulas

Item	Formula
YoY growth	(current ÷ prior) − 1
CAGR	(end ÷ start)^(1/years) − 1
Gross margin %	gross profit ÷ revenue
AOV	revenue ÷ orders
CAC / LTV:CAC / payback	spend ÷ new customers; LTV ÷ CAC; CAC ÷ (LTV ÷ 12)
Retention	customers active in month n ÷ cohort size
Churn	1 − retention
Support / confidence / lift	P(A,B); P(B|A); P(B|A) ÷ P(B)
Precision / recall	TP ÷ (TP+FP); TP ÷ (TP+FN)
MAPE	mean(|forecast − actual| ÷ actual)
Chi-square	Σ (observed − expected)² ÷ expected
Standard error (proportion)	√(p(1−p) ÷ n)
95% CI (proportion)	p ± 1.96 × standard error

20.5 Which chart?

To show...	Use	Avoid
Trend over time	Line (or columns for few periods)	Pie
Compare categories	Sorted horizontal bars	3-D charts
Part-to-whole (few parts)	Stacked bar / treemap	Pie with > 5 slices
Relationship of two numbers	Scatter (bubble for a third)	Dual axes without clear labels
Distribution	Histogram / box plot	Averages alone
Cohort / matrix	Heatmap	Many lines
Single KPI	Card with comparison	Gauge clutter

21. Study plan, project pitches and glossary

21.1 A 14-day study plan

Days	Focus	Output
1-2	Chapters 1-4 (analytics, metrics, data basics, project)	Explain the project in 2 minutes without notes
3-4	Chapters 5-7 (pandas, cleaning, KPIs)	Re-run `01` and `02` scripts; explain every output
5	Chapters 8-9 (RFM, cohorts)	Draw an RFM grid and a cohort table by hand
6-7	Chapter 10 (churn ML)	Explain leakage, AUC, lift with an example
8	Chapters 11-12 (CAC/LTV, basket)	Calculate LTV:CAC and lift on paper
9	Chapters 13-14 (statistics, forecasting)	Run a chi-square by hand; explain MAPE
10-11	Chapter 15 (SQL)	Write 10 window-function queries from memory
12-13	Chapter 16 (Power BI/DAX)	Rebuild three measures; explain filter context aloud
14	Chapters 18-19 (scenarios, Q&A)	Mock interview with a friend

21.2 The 60-second pitch

KEY IDEA

"I built an end-to-end analytics project on a simulated e-commerce retailer: 9,000 customers and 29,000 orders over three years. I cleaned messy raw data with a documented defect log and analysed the same dataset in Python, SQL and Power BI, reconciling the numbers across all three. I did customer segmentation and cohort retention, a churn model with an AUC of 0.83, channel CAC and LTV, statistical testing, and a 6-month forecast. Key findings: discounts did not lift volume but cut margin, so a 10% cap is worth about ₹84 lakh; Paid Search returns 2.5 times its cost versus 12.5 for Referral; and the top 10% of customers drive 49% of revenue."

21.3 The 3-minute structure

1. **Business context** (15 s): who, what question.
2. **Data & cleaning** (20 s): 7 tables, 9 defect types fixed and logged.
3. **Analysis highlights** (90 s): pick 3: pricing, channels, customers; each as finding → number → action.
4. **Technical depth** (30 s): window functions, DAX filter context, leakage-safe churn, backtested forecast, reconciliation.
5. **Limitations & next steps** (25 s): synthetic data, assumptions, A/B test.

21.4 STAR template for behavioural questions

Letter	Say	Length
Situation	One sentence of context	10 s
Task	Your responsibility/goal	10 s
Action	What YOU did (tools, reasoning, trade-offs)	30 s
Result	Quantified outcome + what you learned	15 s

21.5 Things never to say

- 'The model is 90% accurate' without the base rate.
- 'The data showed X caused Y' from observational data.
- 'I cleaned the data' with no detail on what, how many rows, and why that fix.
- 'It's real data' (it is synthetic: always be upfront).
- 'I don't know' without a reasoning path: instead say how you would find out.

21.6 Glossary (A-Z)

Term	Meaning	Term	Meaning
A/B test	Randomised experiment comparing control and treatment	**KPI**	Key performance indicator
ABC analysis	Classifying items by revenue contribution (A top, C tail)	**Leakage**	Future information contaminating model inputs
AOV	Average order value = revenue ÷ orders	**Lift**	Association strength vs chance; also model gain vs random
AUC	Area under the ROC curve; ranking quality (0.5 chance, 1 perfect)	**LTV**	Customer lifetime value
Bootstrap	Resampling method for confidence intervals	**MAPE**	Mean absolute percentage error
Cardinality	Relationship type between tables (1:*, *:*)	**Measure**	DAX calculation evaluated in context
CAC	Customer acquisition cost	**Median**	Middle value
Churn	Customers stopping purchases/subscription	**Null hypothesis**	Default 'no effect' claim
Cohort	Group of customers who started in the same period	**One-hot encoding**	Turning categories into 0/1 columns
Confidence interval	Range expressing precision of an estimate	**Orphan record**	Child row with no parent
Confounder	Hidden factor affecting both cause and effect	**Overfitting**	Model memorises training noise
Context transition	DAX rule turning a row context into a filter context	**Pareto**	80/20 concentration principle
COGS	Cost of goods sold	**p-value**	Probability of results this extreme if the null were true
CTE	Named temporary query result (WITH)	**Power Query**	Data preparation tool using the M language
DAX	Data Analysis Expressions: Power BI formula language	**Primary key**	Unique identifier of a row
Decile / Quintile	10 / 5 equal-sized groups	**Precision / Recall**	Correctness of flagged cases / share of true cases found
Dimension table	Descriptive table (customer, product, date)	**RFM**	Recency, frequency, monetary segmentation
EDA	Exploratory data analysis	**RLS**	Row-level security
Effect size	How big a difference is	**Right-censoring**	Outcome not yet observable for recent cases
ETL	Extract, transform, load	**ROC**	Curve of true vs false positive rate
Fact table	Table of events and measures	**Seasonality**	Repeating calendar pattern
Feature	Input variable for a model	**Silhouette**	Cluster quality score (−1 to 1)

Term	Meaning	Term	Meaning
Fiscal year	Company reporting year (India: Apr-Mar)	**SKU**	Stock keeping unit
Filter context	Rows a DAX calculation can see	**Star schema**	Fact table surrounded by dimension tables
Foreign key	Column pointing to another table's primary key	**Support / Confidence**	Basket metrics: pair frequency / conditional probability
Grain	What one row of a table represents	**View**	Saved query acting as a table
Hypothesis test	Procedure to judge if an effect is real	**Window function**	Calculation across related rows without collapsing them
Index	Structure speeding up database lookups	**YoY / MoM**	Year-over-year / month-over-month change
K-Means	Clustering algorithm using k centroids		

KEY IDEA

You now have the vocabulary, the project, the methods and the framework. In the interview: **slow down, clarify, structure, quantify, and be honest about limits.** That is what a senior analyst sounds like.